

Employee Management API

A step-by-step guide to the code, the reasons behind it, and the journey from an HTTP request to SQL Server and back.

ASP.NET Core 10 • C# • EF Core • Dapper • SQL stored procedures • JWT roles • SMTP email

How to study: read a chapter's explanation, follow its source listing, then predict the result of the exercise before testing it. Source listings use the current backend files. Line numbers refer to those files at the time this document was generated.

This edition expands the earlier `guide_study.pdf` with explanations of statements, method contracts, configuration, data flow, failure paths, and implementation choices. All application C# files outside generated migrations/build output are included. Migration concepts and representative migration code are included separately.

Scope: repository inspection, not a live database or email-delivery certification. SQL examples are teaching implementations compatible with the visible C# contracts; the deployed procedure definitions were not available in this repository. Credentials in examples are placeholders. No application behavior was changed to produce this document.

Contents and Reading Order

1. The complete system and folder structure
2. C# concepts used throughout the project
3. Entities, relationships, and the database context
4. ViewModels, JSON binding, and validation
5. Startup, configuration, and dependency injection
6. Employee controller: every action explained
7. Employee service: all six stored-procedure calls
8. SQL procedures: complete teaching code and EXEC examples
9. Login, password hashing, and JWT creation
10. Manager and Employee authorization
11. Department, Client, and Assigned Task modules
12. SMTP: from settings to employee welcome email
13. Migrations and database deployment
14. How data reaches the frontend
15. Step-by-step setup and API testing
16. Troubleshooting and current implementation gaps
17. Practice lessons and answers
18. File edit map, glossary, and references

What was confirmed in the current code

Feature	Current behavior
Employee persistence	GetByEmail, GetAll, GetById, Create, Update, and Delete all call stored procedures through Dapper.
Other business modules	Department, Client, and Assigned Task services use EF Core.
Login	Configured ApiUser credentials receive a Manager token. Database employee login uses email in the JSON field named username.
Employee update	Profile fields are updated; PasswordHash and Role are copied from the existing employee. The optional request Password and Role are currently ignored.

Feature	Current behavior
Email	After employee creation, the controller awaits an SMTP welcome email when MailSettings.Enabled is true.
Schema startup	Program calls EnsureSchemaAsync, which calls EF Core MigrateAsync.

1. The Complete System

This project is an HTTP API. It receives a request, validates and authorizes it, executes application logic, reads or writes SQL Server data, and returns an HTTP response. A React page can display that response. Postman can call the same endpoints without any page.

```
React form / Postman
-> HTTP request: method + URL + headers + JSON
-> JWT authentication and role authorization
-> Controller action + request ViewModel
-> Service interface resolved to a service object
-> Dapper procedure call OR EF Core query
-> SQL Server
-> C# result object
-> Controller HTTP result
-> JSON response
-> Frontend state -> rendered UI
```

A controller knows about HTTP: route parameters, status codes, and request bodies. A service knows how to obtain and persist domain data. The model represents the stored data. A ViewModel defines what the API accepts or returns. Keeping these responsibilities separate makes the flow easier to test and change.

Folder structure

```
EmployeeApi.sln
EmployeeApi/
  EmployeeApi.csproj
  Program.cs
  EmployeeApiServiceRegistrar.cs
  appsettings.json
  appsettings.Development.json
  Properties/launchSettings.json
  Api/v1/
    AuthApiController.cs
    EmployeeApiController.cs
    DepartmentApiController.cs
    ClientApiController.cs
    AssignTaskApiController.cs
  Configuration/MailSettings.cs
  Data/
    ApplicationDbContext.cs
    EmployeeApiDatabaseInitializer.cs
  Model/
    Employee/Employee.cs
    Department/Department.cs
    Client/Client.cs
    AssignTask/AssignedTask.cs
  ViewModel/
    Auth/      LoginRequestViewModel, LoginResponseViewModel
```

```

Employee/    CreateEmployeeViewModel, UpdateEmployeeViewModel
Department/ CreateDepartmentViewModel, UpdateDepartmentViewModel
Client/      CreateClientViewModel, UpdateClientViewModel
AssignTask/ CreateAssignTaskViewModel, UpdateAssignTaskViewModel
Service/
Employee/    IEmployeeService, EmployeeService
Department/ IDepartmentService, DepartmentService
Client/      IClientService, ClientService
AssignTask/ IAssignTaskService, AssignTaskService
Notifications/
    IEmployeeEmailSender.cs
    SmtplibEmployeeEmailSender.cs
Migrations/
    timestamped migration and Designer files
    ApplicationDbContextModelSnapshot.cs
Postman/
    EmployeeApi.postman_collection.json
    EmployeeApi.postman_environment.json
Documents/
    guide_study.html / guide_study.pdf
    Syllabus.html / Syllabus.pdf
    firststeptoread.pdf
    final_doc.html / final_doc.pdf

```

The folder named `Api/v1` does not automatically put `/v1` in a URL. Route attributes define the URLs. The employee controller uses `[Route("api/employee")]`, so the URL begins with `/api/employee`.

There is no Razor View in this flow

The controllers inherit `ControllerBase` and return results such as `Ok(employee)`. They do not call `View(employee)`. ASP.NET Core serializes the object to JSON. The frontend receives that JSON and renders a component. The backend folder named `ViewModel` contains data contracts, not HTML templates.

Study task: trace `GET /api/employee/5`. Which file receives it? Which service method is called? Which SQL procedure name is sent? Answer: `EmployeeApiController.GetById`, `EmployeeService.GetById`, `dbo.GetEmployeeById`.

2. C# Concepts You Need

Code	Meaning in this project
<code>namespace EmployeeApi.Service.Employee</code>	Groups type names. A namespace helps distinguish an Employee model from similarly named folders or types.
<code>using EmployeeEntity = ...Employee;</code>	An alias: EmployeeEntity is another name for the existing Employee class. It does not create a new class or database table.
<code>public class</code>	A type that can be used by other code. Creating an instance gives you an object with that type's state and behavior.
<code>private readonly IEmployeeService _service;</code>	A field accessible inside the class. Its reference can be assigned during construction and is then kept for the object's lifetime. Readonly does not make the service object immutable.
<code>public string Name { get; set; }</code>	A property that can be read and assigned. JSON binding and database mapping can assign values to it.
<code>= string.Empty</code>	The initial string is empty. This avoids an initial null value; it does not satisfy Required validation for a request.
<code>EmployeeEntity? / int?</code>	The result or value may be null. GetById can return no employee; ClientId can represent no client.
<code>null!</code>	Suppresses a compiler nullability warning. It neither creates an object nor guarantees that the runtime value is non-null.
<code>async Task<EmployeeEntity></code>	The method returns an asynchronous operation whose eventual result is an employee. It does not mean a new thread is started for every call.
<code>await</code>	Waits for an asynchronous operation to complete without holding a thread during the asynchronous I/O wait. Exceptions still propagate to the caller.
<code>using var connection = ...</code>	Ensures disposal at the end of the scope, including when an exception occurs. SqlConnection disposal normally returns the connection to its pool.
<code>new { Id = id }</code>	An anonymous object. Dapper uses the property name Id as the SQL parameter name and the value id as its value.

Code	Meaning in this project
<code>employee is null ? NotFound() : Ok(employee)</code>	A conditional expression. No object produces 404; an object produces 200 with a body.
<code>value ?? throw ...</code>	Use the value if it is non-null; otherwise throw the exception. An empty string is not null and will not trigger this check.
<code>e => e.Id == id</code>	A lambda expression. In an EF query it describes a condition that EF can translate into SQL.
<code>[HttpPost], [Required]</code>	Attributes add metadata. Different framework components read that metadata to route a request or validate its values.

Interface versus implementation

`IEmployeeService` is a contract: it lists methods and their result types. `EmployeeService` supplies the method bodies. A controller asks for the interface in its constructor. Dependency injection chooses the implementation registered in the service container. You do not call the interface and implementation separately; the interface reference points to an implementation object.

```
IEmployeeService service = an EmployeeService instance;
await service.GetById(5);
// Executes EmployeeService.GetById(5).
```

Entity, table row, and JSON are different things

An `Employee` object lives in application memory. A row lives in SQL Server. Dapper can map a result row into the object. A JSON serializer can later turn selected object properties into a response. Attributes such as `JsonIgnore` affect serialization; they do not remove the corresponding column from SQL Server.

3. Entities and Database Context

Employee

Current source: EmployeeApi/Model/Employee/Employee.cs. Original line numbers; comments omitted where applicable.

```
2 using System.Text.Json.Serialization;
3
4 namespace EmployeeApi.Model.Employee
5 {
6     public class Employee
7     {
8         public int Id { get; set; }
9
10        public string Name { get; set; } = string.Empty;
11
12        public string Email { get; set; } = string.Empty;
13
14        public string PhoneNumber { get; set; } = string.Empty;
15
16        public decimal Salary { get; set; }
17
18        public int DepartmentId { get; set; }
19
20        public int? ClientId { get; set; }
21
22        [JsonIgnore]
23        public string PasswordHash { get; set; } = string.Empty;
24        public string Role { get; set; } = "Employee";
25        public EmployeeApi.Model.Client.Client? Client { get; set; }
26
27        public EmployeeApi.Model.Department.Department? Department { get; set; }
28    }
29 }
30
31
32
```

Property or statement	Explanation
Id	Primary key by EF convention. SQL Server identity generation supplies it for a newly inserted row in the migration model.
Name, Email, PhoneNumber	Profile information. Email is also the database login lookup value. A Name value is not used by the current login lookup.
decimal Salary	A decimal value for money. ApplicationDbContext configures SQL precision 18,2.
DepartmentId and Department	DepartmentId is the stored foreign-key value; Department is a reference to a related C# object. A navigation property is a relationship, not inheritance.

Property or statement	Explanation
int? ClientId and Client?	An employee can have no client. The nullable reference allows the related Client object to be absent or not loaded.
[JsonIgnore] PasswordHash	The hash stays available to C# and SQL mapping but is excluded from the normal JSON representation. It must be present in the login query's result for password verification.
Role = "Employee"	A C# default used when a new object is constructed. It does not detect seniority or automatically classify a manager. Create explicitly replaces it with model.Role, and Dapper replaces it when a Role column is returned.

A manager is stored as an employee record whose Role is Manager, or can be the separate configured login that explicitly receives a Manager token. There is no separate Manager entity or Admin role implementation in these files.

Department, Client, and Assigned Task

Current source: EmployeeApi/Model/Department/Department.cs. Original line numbers; comments omitted where applicable.

```

1 using EmployeeEntity = EmployeeApi.Model.Employee.Employee;
2
3 namespace EmployeeApi.Model.Department
4 {
5     public class Department
6     {
7         public int Id { get; set; }
8
9         public string Name { get; set; } = string.Empty;
10
11        public string Description { get; set; } = string.Empty;
12
13        public ICollection<EmployeeEntity> Employees { get; set; }
14        = new List<EmployeeEntity>();
15
16    }
17 }
18 }
19

```

ICollection<EmployeeEntity> Employees represents the many employees associated with a department. Initializing it with a list means code can add items without first constructing a collection. It does not load rows from SQL Server.

Current source: EmployeeApi/Model/Client/Client.cs. Original line numbers; comments omitted where applicable.

```

1 namespace EmployeeApi.Model.Client
2 {
3     public class Client

```

```

4    {
5        public int Id { get; set; }
6
7        public string ClientName { get; set; } = string.Empty;
8
9        public string PhoneNumber { get; set; } = string.Empty;
10
11       public string ProjectName { get; set; } = string.Empty;
12    }
13 }
14

```

ClientName identifies the customer, PhoneNumber stores contact information, and ProjectName describes the project. The visible model is intentionally small: it does not have login credentials or its own authorization role.

Current source: EmployeeApi/Model/AssignTask/AssignedTask.cs. Original line numbers; comments omitted where applicable.

```

1 namespace EmployeeApi.Model.AssignTask;
2
3 public class AssignedTask
4 {
5     public int Id { get; set; }
6
7     public int EmployeeId { get; set; }
8
9     public string Title { get; set; } = string.Empty;
10
11    public string Description { get; set; } = string.Empty;
12
13    public DateTime? DueDate { get; set; }
14
15    public string Status { get; set; } = "Pending";
16
17    public DateTime AssignedOn { get; set; } = DateTime.UtcNow;
18 }
19

```

EmployeeId identifies the intended employee; Title, Description, DueDate, and Status describe the assignment. DueDate is optional. AssignedOn gets a UTC timestamp when a new object is created. Status begins as Pending unless another value is assigned.

Relationship detail: AssignedTask contains an EmployeeId integer, but the inspected EF snapshot does not define an AssignedTask-to-Employee foreign key. The service checks employee existence before create/update. That application check is not equivalent to a database foreign-key constraint and can race with deletion.

ApplicationDbContext

Current source: EmployeeApi/Data/ApplicationDbContext.cs. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Model.AssignTask;
2 using EmployeeApi.Model.Client;

```

```

3 using EmployeeApi.Model.Department;
4 using EmployeeApi.Model.Employee;
5 using Microsoft.EntityFrameworkCore;
6
7 namespace EmployeeApi.Data
8 {
9     public class ApplicationDbContext : DbContext
10    {
11        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
12            : base(options)
13        {
14        }
15
16        public DbSet<Employee> Employees { get; set; } = null!;
17
18        public DbSet<AssignedTask> AssignedTasks { get; set; } = null!;
19
20        public DbSet<Department> Departments { get; set; } = null!;
21
22        public DbSet<Client> Clients { get; set; } = null!;
23
24        protected override void OnModelCreating(ModelBuilder modelBuilder)
25        {
26            modelBuilder.Entity<Employee>()
27                .Property(e => e.Salary)
28                .HasPrecision(18, 2);
29
30            modelBuilder.Entity<Employee>()
31                .HasOne(e => e.Department)
32                .WithMany(d => d.Employees)
33                .HasForeignKey(e => e.DepartmentId);
34
35            base.OnModelCreating(modelBuilder);
36        }
37    }
38 }
39

```

1. The constructor receives configured DbContextOptions and passes them to the base DbContext. Program provides SQL Server connection options through DI.
2. Each DbSet exposes query and change-tracking operations for its entity type. A DbSet is not an already loaded list of every row.
3. OnModelCreating adds database mapping rules beyond conventions.
4. HasPrecision(18, 2) specifies 18 total decimal digits, including 2 after the decimal point.
5. HasOne, WithMany, and HasForeignKey describe the Employee-to-Department relationship.
6. The base method is called after the custom configuration.

The current snapshot records a required Department relationship with cascade deletion and an optional Client relationship. Treat department deletion carefully: the configured database relationship can delete its employees. Check the actual database constraints before relying on any deletion behavior.

EF Core can load related objects using Include. Dapper's single-type QueryAsync<Employee> does not automatically build Department or Client objects, even if SQL joins their tables. A flattened response DTO or Dapper multi-mapping is needed when related details must be returned.

4. ViewModels and Validation

When a request arrives, ASP.NET Core deserializes its JSON into the action parameter type. For example, the Create action receives a CreateEmployeeViewModel. With ApiController enabled, validation errors normally produce a 400 response before the action body executes.

Create employee request

Current source: EmployeeApi/ViewModel/Employee/CreateEmployeeViewModel.cs. Original line numbers; comments omitted where applicable.

```
3 using System.ComponentModel.DataAnnotations;
4
5 namespace EmployeeApi.ViewModel.Employee;
6
7 public class CreateEmployeeViewModel
8 {
9     [Required]
10    [StringLength(150)]
11    public string Name { get; set; } = string.Empty;
12
13    [Required]
14    [EmailAddress]
15    [StringLength(255)]
16    public string Email { get; set; } = string.Empty;
17
18    [StringLength(10)]
19    [Phone]
20    public string? PhoneNumber { get; set; }
21
22    [Range(0, double.MaxValue)]
23    public decimal Salary { get; set; }
24
25    public int? ClientId { get; set; }
26
27    public int DepartmentId { get; set; }
28
29    [Required]
30    public string Password { get; set; } = string.Empty;
31    [Required]
32    public string Role { get; set; } = "Employee";
33
34 }
35
```

Attribute or field	What it actually enforces
[Required]	Rejects missing/null and invalid empty string input for these string properties. It does not check database uniqueness.

Attribute or field	What it actually enforces
[StringLength(150)] Name	Limits the incoming name length. Putting this attribute on a request ViewModel does not automatically change the Employee table column type.
[EmailAddress]	Checks email address format. It does not verify mailbox ownership or that the mailbox exists.
[StringLength(10)] [Phone]	Caps the input at 10 characters and applies a phone-format check. It does not mean exactly 10 digits, and can exclude international numbers.
[Range(0, double.MaxValue)] Salary	Rejects negative input. SQL decimal(18,2) has a smaller maximum, so not every value accepted by this attribute fits the database.
DepartmentId	No positive-ID validation is present here. A nonexistent department will normally fail the database foreign key.
Password	Required on create, but no create-time length or complexity rule is present.
Role	Required with a default Employee value. There is no allowlist limiting input to Manager or Employee.

Update employee request

Current source: EmployeeApi/ViewModel/Employee/UpdateEmployeeViewModel.cs. Original line numbers; comments omitted where applicable.

```

2 using System.ComponentModel.DataAnnotations;
3
4 namespace EmployeeApi.ViewModel.Employee;
5
6 public class UpdateEmployeeViewModel
7 {
8     [Required]
9     [StringLength(150)]
10    public string Name { get; set; } = string.Empty;
11
12    [Required]
13    [EmailAddress]
14    [StringLength(255)]
15    public string Email { get; set; } = string.Empty;
16
17    [StringLength(10)]
18    [Phone]
19    public string? PhoneNumber { get; set; }
20
21    [StringLength(100, MinimumLength = 6)]
22    public string? Password { get; set; }
23
24    public string? Role { get; set; }

```

```

25
26     [Range(0, double.MaxValue)]
27     public decimal Salary { get; set; }
28
29     public int? ClientId { get; set; }
30
31     public int DepartmentId { get; set; }
32 }
33

```

Create and Update use separate types because their contracts can differ. Update's Password is optional and has a 6-to-100 character rule when supplied. Role is optional. However, **the current Update action does not use either field**. Having a property on a ViewModel only allows binding and validation; application code must read it for it to affect the employee.

Profile values such as Name and Email remain required on update. This endpoint behaves as a replacement of the listed profile fields, not a PATCH that changes only fields supplied in JSON.

Login contracts

Current source: EmployeeApi/ViewModel/Auth/LoginRequestViewModel.cs. Original line numbers; comments omitted where applicable.

```

1 namespace EmployeeApi.ViewModel.Auth;
2
3 public class LoginRequestViewModel
4 {
5     public string Username { get; set; } = string.Empty;
6
7     public string Password { get; set; } = string.Empty;
8 }
9

```

Current source: EmployeeApi/ViewModel/Auth/LoginResponseViewModel.cs. Original line numbers; comments omitted where applicable.

```

1 namespace EmployeeApi.ViewModel.Auth;
2
3 public class LoginResponseViewModel
4 {
5     public string Token { get; set; } = string.Empty;
6
7     public DateTime ExpiresAt { get; set; }
8 }
9

```

Username is the JSON input field name, even when its value is an employee email. Password carries the supplied password for verification. The response contains Token and ExpiresAt. The web JSON defaults serialize these as token and expiresAt. There is no separate Role property in LoginResponseViewModel; the token contains the role claim.

The login request has no explicit DataAnnotations validation rules. Empty default strings are possible, so input validation should be considered separately from password checking.

Predict: a Manager sends an employee update containing "role": "Manager" for an existing Employee. Will that employee become a Manager? Answer: no, the current controller copies existingEmployee.Role instead of using model.Role.

5. Startup, Configuration, and DI

Project dependencies

Current source: EmployeeApi/EmployeeApi.csproj. Original line numbers; comments omitted where applicable.

```
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2
3   <PropertyGroup>
4     <TargetFramework>net10.0</TargetFramework>
5     <Nullable>enable</Nullable>
6     <ImplicitUsings>enable</ImplicitUsings>
7   </PropertyGroup>
8
9   <ItemGroup>
10    <PackageReference Include="Dapper" Version="2.1.79" />
11    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.0" />
12    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.0" />
13    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.9">
14      <PrivateAssets>all</PrivateAssets>
15      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
16    </PackageReference>
17    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
18  </ItemGroup>
19
20 </Project>
21
```

The Web SDK enables ASP.NET Core build behavior. TargetFramework selects .NET 10. Nullable enables nullable reference analysis; ImplicitUsings adds common imports automatically. Dapper maps SQL results; the JWT bearer package validates tokens; EF Core SQL Server provides database access and schema migrations; EF Tools supports migration tooling; Swashbuckle exposes Swagger API documentation.

Safe example configuration

Edit location: EmployeeApi/appsettings.json, with environment-specific overrides or a secret store.

This is a teaching example, not a copy of the local credentials. Merge the relevant sections instead of replacing unrelated configuration. The SQL connection uses Windows authentication; change it to match your own SQL Server environment.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=YOUR_SERVER;Database=YOUR_DATABASE;Trusted_Connection=True;Encrypt=True;TrustServerCertificate=True"
  },
  "Jwt": {
    "Secret": "REPLACE_WITH_A_RANDOM_SECRET_OF_AT_LEAST_32_BYTES",
    "Issuer": "EmployeeApi",
    "Audience": "EmployeeFrontend",
  }
}
```

```

    "ExpirationMinutes": 30
  },
  "ApiUser": {
    "Username": "temporary-manager",
    "Password": "REPLACE_WITH_A_LOCAL_DEVELOPMENT_PASSWORD"
  },
  "MailSettings": {
    "Enabled": false,
    "Host": "smtp.gmail.com",
    "Port": 587,
    "UseSsl": true,
    "UserName": "your-sender@gmail.com",
    "Password": "YOUR_PROVIDER_APP_PASSWORD",
    "FromEmail": "your-sender@gmail.com",
    "FromName": "Employee Management"
  }
}

```

The colon syntax selects a nested configuration value: `Jwt:Secret` selects `Secret` inside `Jwt`. `GetConnectionString("DefaultConnection")` selects `ConnectionStrings:DefaultConnection`. `Configure<MailSettings>` binds properties from the `MailSettings` section to a typed options object.

The default application builder reads `appsettings.json` and the environment-specific settings file, with higher-priority providers such as environment variables able to override values. An environment variable such as `MailSettings__Password` represents `MailSettings:Password`. The inspected `appsettings.Development.json` contains logging configuration.

`TrustServerCertificate=True` bypasses certificate trust validation for that SQL connection. It is useful for a local development server; production should use a properly trusted SQL Server certificate. Do not treat the example signing secret or passwords as deployable credentials.

Program.cs: complete startup code

Current source: `EmployeeApi/Program.cs`. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Data;
2 using EmployeeApi;
3 using EmployeeApi.Configuration;
4 using Microsoft.AspNetCore.Authentication.JwtBearer;
5 using Microsoft.EntityFrameworkCore;
6 using Microsoft.IdentityModel.Tokens;
7 using Microsoft.OpenApi.Models;
8 using System.Text;
9 using System.Text.Json.Serialization;
10
11 var builder = WebApplication.CreateBuilder(args);
12
13 var jwtSecret = builder.Configuration["Jwt:Secret"]
14     ?? throw new InvalidOperationException("Jwt:Secret is not configured.");
15
16 builder.Services.AddControllers()
17     .AddJsonOptions(options =>
18     {
19         options.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;

```

```

20     });
21 builder.Services.AddDbContext<ApplicationDbContext>(options =>
22     options.UseSqlServer(
23         builder.Configuration.GetConnectionString("DefaultConnection")));
24 builder.Services.Configure<MailSettings>(builder.Configuration.GetSection("MailSettings"));
25 builder.Services.AddEmployeeApiServices();
26 builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
27     .AddJwtBearer(options =>
28     {
29         options.TokenValidationParameters = new TokenValidationParameters
30         {
31             ValidateIssuer = true,
32             ValidateAudience = true,
33             ValidateLifetime = true,
34             ValidateIssuerSigningKey = true,
35             ValidIssuer = builder.Configuration["Jwt:Issuer"],
36             ValidAudience = builder.Configuration["Jwt:Audience"],
37             IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSecret)),
38             ClockSkew = TimeSpan.Zero
39         };
40     });
41 builder.Services.AddAuthorization();
42 builder.Services.AddEndpointsApiExplorer();
43 builder.Services.AddSwaggerGen(options =>
44 {
45     options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
46     {
47         Name = "Authorization",
48         Type = SecuritySchemeType.Http,
49         Scheme = "bearer",
50         BearerFormat = "JWT",
51         In = ParameterLocation.Header,
52         Description = "Enter your JWT bearer token."
53     });
54     options.AddSecurityRequirement(new OpenApiSecurityRequirement
55     {
56         {
57             new OpenApiSecurityScheme
58             {
59                 Reference = new OpenApiReference
60                 {
61                     Type = ReferenceType.SecurityScheme,
62                     Id = "Bearer"
63                 }
64             },
65             Array.Empty<string>()
66         }
67     });
68 });
69 builder.Services.AddCors(options =>
70 {
71     options.AddPolicy("ReactPolicy", policy =>
72     {
73         policy.AllowAnyOrigin()
74             .AllowAnyHeader()
75             .AllowAnyMethod();
76     });
77 });
78
79 var app = builder.Build();
80
81 using (var scope = app.Services.CreateScope())

```

```

82 {
83     var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
84     await EmployeeApiDatabaseInitializer.EnsureSchemaAsync(dbContext);
85 }
86
87 if (app.Environment.IsDevelopment())
88 {
89     app.UseSwagger();
90     app.UseSwaggerUI();
91 }
92
93 if (!app.Environment.IsDevelopment())
94 {
95     app.UseHttpsRedirection();
96 }
97
98 app.UseCors("ReactPolicy");
99 app.UseAuthentication();
100 app.UseAuthorization();
101 app.MapControllers();
102
103 app.Run();
104

```

Statement or block	Why it is here
WebApplication.CreateBuilder(args)	Creates configuration, logging, hosting, and the dependency injection container builder.
Read Jwt:Secret	Token validation needs the same signing key used when creating tokens. A missing value stops startup; an empty value is not rejected by this null check.
AddControllers + IgnoreCycles	Registers API controllers and configures serialization to avoid infinite cycles from navigation properties. It does not load navigation properties.
AddDbContext + UseSqlServer	Registers an EF context per request scope, configured for SQL Server.
Configure<MailSettings>	Binds the JSON section for later injection as IOptions<MailSettings>.
AddEmployeeApiServices	Calls the project's extension method that registers service interfaces and implementations.
AddAuthentication + AddJwtBearer	Selects bearer authentication and defines the rules for validating an incoming JWT.
TokenValidationParameters	Checks issuer, audience, expiry, and signature. ClockSkew.Zero means no extra expiry grace period is configured.

Statement or block	Why it is here
AddAuthorization	Registers the authorization services used to enforce Authorize attributes.
AddSwaggerGen	Documents endpoints and describes bearer authentication for Swagger's Authorize control. Documentation settings do not themselves secure an endpoint.
AddCors ReactPolicy	Allows browser cross-origin requests from any origin with any method/header. CORS is a browser rule, not a replacement for authentication.
builder.Build()	Builds the application so the request pipeline can be configured.
CreateScope + EnsureSchemaAsync	Resolves a scoped database context outside a request and applies pending migrations before the application starts serving.
Development condition	Swagger is exposed in Development. HTTPS redirection is enabled outside Development.
UseCors; UseAuthentication; UseAuthorization	Applies browser access rules, establishes identity, then checks access requirements. Authentication must precede authorization.
MapControllers; Run	Maps controller routes, then runs the host to receive requests.

Service registration

Current source: EmployeeApi/EmployeeApiServiceRegistrar.cs. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Service.AssignTask;
2 using EmployeeApi.Service.Client;
3 using EmployeeApi.Service.Department;
4 using EmployeeApi.Service.Employee;
5 using EmployeeApi.Service.Notifications;
6
7 namespace EmployeeApi;
8
9 public static class EmployeeApiServiceRegistrar
10 {
11     public static IServiceCollection AddEmployeeApiServices(this IServiceCollection services)
12     {
13         services.AddScoped<IEmployeeService, EmployeeService>();
14         services.AddScoped<IAssignTaskService, AssignTaskService>();
15
16         services.AddScoped<IDepartmentService, DepartmentService>();
17         services.AddScoped<IClientService, ClientService>();
18         services.AddScoped<IEmployeeEmailSender, SmtEmployeeEmailSender>();

```

```
19
20     return services;
21 }
22 }
23
```

This is an extension method because its first parameter is this `IServiceCollection services`. That syntax allows Program to call `builder.Services.AddEmployeeApiServices()`. Each `AddScoped` line says which implementation to construct when a consumer asks for that interface.

Scoped means an instance is shared within a scope, typically one HTTP request. A different request gets a different scoped instance. In `EmployeeApiController`'s constructor, DI supplies both `EmployeeService` and `SmtpEmployeeEmailSender`, and supplies their own dependencies recursively.

Predict: remove the `IEmployeeEmailSender` registration in a study branch. What fails? The container cannot construct `EmployeeApiController` when resolving the missing dependency. Installing an SMTP package alone does not register this interface.

6. Employee Controller Explained

Current source: EmployeeApi/Api/v1/EmployeeApiController.cs. Original line numbers; comments omitted where applicable.

```
2 using Microsoft.AspNetCore.Identity;
3 using EmployeeApi.Service.Employee;
4 using EmployeeApi.Service.Notifications;
5 using EmployeeApi.ViewModel.Employee;
6 using Microsoft.AspNetCore.Mvc;
7 using EmployeeEntity = EmployeeApi.Model.Employee.Employee;
8 using Microsoft.AspNetCore.Authorization;
9
10 namespace EmployeeApi.Api.v1 {
11     [ApiController]
12     [Route("api/employee")]
13     public class EmployeeApiController : ControllerBase
14     {
15         private readonly IEmployeeService _service;
16         private readonly IEmployeeEmailSender _employeeEmailSender;
17
18         public EmployeeApiController(IEmployeeService service, IEmployeeEmailSender employeeEmailSender)
19         {
20             _service = service;
21             _employeeEmailSender = employeeEmailSender;
22         }
23
24         [HttpGet]
25         [Authorize(Roles = "Manager,Employee")]
26         public async Task<IActionResult> Get()
27         {
28             return Ok(await _service.GetAll());
29         }
30
31         [HttpGet("{id:int}")]
32         [Authorize(Roles = "Manager,Employee")]
33         public async Task<IActionResult> GetById(int id)
34         {
35             var employee = await _service.GetById(id);
36             return employee is null ? NotFound() : Ok(employee);
37         }
38
39         [HttpPost("new")]
40         [Authorize(Roles = "Manager")]
41         public async Task<IActionResult> Create([FromBody] CreateEmployeeViewModel model)
42         {
43             var employee = new EmployeeEntity
44             {
45                 Name = model.Name,
46                 Email = model.Email,
47                 PhoneNumber = model.PhoneNumber,
48                 Salary = model.Salary,
49                 ClientId = model.ClientId,
50                 DepartmentId = model.DepartmentId
51             };
52
53             var hasher = new PasswordHasher<EmployeeEntity>();
54
55             employee.PasswordHash = hasher.HashPassword(employee, model.Password);
```

```

56         employee.Role = model.Role;
57
58         var createdEmployee = await _service.Create(employee);
59         await _employeeEmailSender.SendAccountCreatedAsync(createdEmployee, model.Password);
60
61         return CreatedAtAction(nameof(GetById), new { id = createdEmployee.Id }, createdEmployee);
62     }
63
64     [HttpPut("{id:int}")]
65     [Authorize(Roles = "Manager")]
66     public async Task<IActionResult> Update(int id, [FromBody] UpdateEmployeeViewModel model)
67     {
68         var existingEmployee = await _service.GetById(id);
69         if (existingEmployee is null)
70         {
71             return NotFound();
72         }
73
74         var employee = new EmployeeEntity
75         {
76             Id = id,
77             Name = model.Name,
78             Email = model.Email,
79             PhoneNumber = model.PhoneNumber,
80             PasswordHash = existingEmployee.PasswordHash,
81             Role = existingEmployee.Role,
82             Salary = model.Salary,
83             ClientId = model.ClientId,
84             DepartmentId = model.DepartmentId
85         };
86
87         return Ok(await _service.Update(employee));
88     }
89
90     [HttpDelete("{id:int}")]
91     [Authorize(Roles = "Manager")]
92     public async Task<IActionResult> Delete(int id)
93     {
94         return await _service.Delete(id) ? NoContent() : NotFound();
95     }
96 }
97 }
98

```

Routing and construction

ApiController enables API binding and validation conventions. Route gives the common URL prefix. ControllerBase provides Ok, NotFound, Unauthorized, NoContent, and CreatedAtAction helpers. The constructor stores injected service references so actions can call them.

[HttpGet("{id:int}")] matches a numeric route segment, such as /api/employee/12. The route value is bound to int id. It does not prove that row 12 exists.

Get: list employees

1. The role attribute permits Manager or Employee.

2. GetAll returns an asynchronous list from the employee service.
3. Ok wraps the list in HTTP 200. An empty list is a successful result, normally serialized as [] rather than 404.

There is no current-user filter in this action or in the C# GetAll method. Role access is not the same as permission to read only one's own row. Salary and other public entity properties are included in the response; only PasswordHash is explicitly hidden.

GetById: one employee

The service returns either an Employee object or null. The conditional return chooses 404 for null and 200 for an object. Any employee allowed by the role attribute can request any numeric ID; no employee-ownership comparison is present.

Create: request to saved account

1. [HttpPost("new")] makes the URL /api/employee/new. The Manager role is required.
2. [FromBody] CreateEmployeeViewModel model receives validated JSON values.
3. new EmployeeEntity { ... } maps allowed profile fields from the request into an entity. Id is left for SQL Server to generate.
4. new PasswordHasher<EmployeeEntity>() creates the framework password hashing helper.
5. HashPassword(employee, model.Password) produces an encoded salted hash. The raw password is not assigned to a database entity property.
6. employee.Role = model.Role selects the stored role from the authorized Manager's request. The current ViewModel does not restrict its values to an allowlist.
7. await _service.Create(employee) sends entity values, including PasswordHash, to dbo.CreateEmployee and receives the inserted row.
8. SendAccountCreatedAsync(createdEmployee, model.Password) sends the welcome email using the original password still available in the request.
9. CreatedAtAction(nameof(GetById), ...) produces HTTP 201, a body containing the created employee, and a Location header pointing to its GET URL. It does not execute GetById and does not redirect the frontend to a dashboard.

Two separate side effects: the row is saved before email is sent. There is no transaction spanning SQL and SMTP, and no catch here for SMTP failure. If SMTP throws, the request can fail with 500 even though the employee exists. Check for that row before retrying creation.

Update: what changes and what stays

1. The Manager calls PUT /api/employee/{id} with the update body.
2. The action first loads the existing employee. Missing data returns 404 before any update.

3. A new entity is built with Id from the URL and the profile values from the body.
4. PasswordHash and Role are copied from existingEmployee. This avoids replacing them with empty defaults, but also means model.Password and model.Role have no effect.
5. The service passes all mapped values to dbo.UpdateEmployee. The result is returned with 200.

To later support a password or role change, the edit location is this mapping block: validate the role against allowed values, hash a supplied nonblank password, and preserve existing values when not changing them. A self-service password change would need its own authorized contract and current-password check. That feature is not implemented by the existing optional properties.

A row can disappear between the existence check and the update. The service then throws `KeyNotFoundException`, which this action does not catch. The preliminary check does not make the operation atomic.

Delete: bool to HTTP status

The Manager calls `DELETE /api/employee/{id}`. The service returns true when the procedure's scalar deletion count is greater than zero. True becomes 204 with no response body; false becomes 404. SQL exceptions still propagate; false does not mean every possible SQL failure was handled.

7. Employee Service and Dapper

The service contract

Current source: EmployeeApi/Service/Employee/IEmployeeService.cs. Original line numbers; comments omitted where applicable.

```
3 using EmployeeEntity = EmployeeApi.Model.Employee.Employee;
4
5 namespace EmployeeApi.Service.Employee
6 {
7     public interface IEmployeeService
8     {
9         Task<List<EmployeeEntity>> GetAll();
10
11         Task<EmployeeEntity?> GetById(int id);
12
13         Task<EmployeeEntity> Create(EmployeeEntity employee);
14
15         Task<EmployeeEntity> Update(EmployeeEntity employee);
16
17         Task<bool> Delete(int id);
18
19         Task<EmployeeEntity?> GetByEmail(string email);
20     }
21 }
22
```

The interface makes all six operations available to a controller or another service. List retrieval returns `List<EmployeeEntity>`. Single-row lookups return nullable employees because absence is normal. Create must return an employee; Update either returns an employee or throws; Delete returns a bool.

The complete current implementation

Current source: EmployeeApi/Service/Employee/EmployeeService.cs. Original line numbers; comments omitted where applicable.

```
2 using Dapper;
3 using Microsoft.Data.SqlClient;
4 using System.Data;
5 using EmployeeApi.Data;
6 using Microsoft.EntityFrameworkCore;
7 using EmployeeEntity = EmployeeApi.Model.Employee.Employee;
8
9 namespace EmployeeApi.Service.Employee
10 {
11     public class EmployeeService : IEmployeeService
12     {
13         private readonly ApplicationDbContext _context;
14
15         private readonly string _connString;
16         public EmployeeService(ApplicationDbContext context, IConfiguration configuration)
17         {
```

```

18         _context = context;
19         _connString = configuration.GetConnectionString("DefaultConnection") ??
20             throw new InvalidOperationException("Connection string not found.");
21     }
22
23     public async Task<EmployeeEntity?> GetByEmail(string email)
24     {
25         using var connection = new SqlConnection(_connString);
26         return await connection.QueryFirstOrDefaultAsync<EmployeeEntity>(
27             "[dbo].[GetEmployeeByEmail]",
28             new
29             {
30                 Email= email
31             },
32             commandType: CommandType.StoredProcedure);
33     }
34
35     public async Task<List<EmployeeEntity>> GetAll()
36     {
37         using var connection = new SqlConnection(_connString);
38
39         var employees = await connection.QueryAsync<EmployeeEntity>(
40             "[dbo].[GetAllEmployees]",
41             commandType: CommandType.StoredProcedure);
42
43         return employees.ToList();
44     }
45
46     public async Task<EmployeeEntity?> GetById(int id)
47     {
48         using var connection = new SqlConnection(_connString);
49
50         return await connection.QueryFirstOrDefaultAsync<EmployeeEntity>(
51             "[dbo].[GetEmployeeById]",
52             new { Id = id },
53             commandType: CommandType.StoredProcedure);
54     }
55
56     public async Task<EmployeeEntity> Create(EmployeeEntity employee)
57     {
58         using var connection = new SqlConnection(_connString);
59
60         var createdEmployee = await connection.QuerySingleAsync<EmployeeEntity>(
61             "[dbo].[CreateEmployee]",
62             new
63             {
64                 employee.Name,
65                 employee.Email,
66                 employee.PhoneNumber,
67                 employee.PasswordHash,
68                 employee.Role,
69                 employee.Salary,
70                 employee.DepartmentId,
71                 employee.ClientId
72             },
73             commandType: CommandType.StoredProcedure);
74
75         return createdEmployee;
76     }
77
78     public async Task<EmployeeEntity> Update(EmployeeEntity employee)
79     {

```

```

113         using var connection = new SqlConnection(_connString);
114
115         var updatedEmployee = await connection.QueryFirstOrDefaultAsync<EmployeeEntity>(
116             "[dbo].[UpdateEmployee]",
117             new
118             {
119                 employee.Id,
120                 employee.Name,
121                 employee.Email,
122                 employee.PhoneNumber,
123                 employee.PasswordHash,
124                 employee.Role,
125                 employee.Salary,
126                 employee.DepartmentId,
127                 employee.ClientId
128             },
129             commandType: CommandType.StoredProcedure);
130
131         return updatedEmployee
132             ?? throw new KeyNotFoundException($"Employee {employee.Id} was not found.");
133     }
134
135     public async Task<bool> Delete(int id)
136     {
137         using var connection = new SqlConnection(_connString);
138
139         var affectedRows = await connection.ExecuteScalarAsync<int>(
140             "[dbo].[DeleteEmployee]",
141             new { Id = id },
142             commandType: CommandType.StoredProcedure);
143
144         return affectedRows > 0;
145     }
146 }
147
148 }
149
150 }
151
152 }
153
154 }
155
156 }
157
158 }
159
160 }
161

```

Older commented EF implementations are omitted from the study listing to make the active path easier to follow. Original source line numbers are retained. All six active employee methods now use Dapper. The injected `ApplicationDbContext` field and EF import remain, but the active employee operations do not use that field.

Constructor, connection, and parameter mapping

The constructor reads `DefaultConnection` once and stores it in `_connString`. A missing connection string throws a configuration error. Each operation constructs a `SqlConnection`. Dapper's asynchronous APIs open a closed connection as needed for execution; the `using` declaration ensures disposal afterward. You can explicitly `OpenAsync`, but its absence here is not the cause of a missing-procedure error.

`CommandType.StoredProcedure` tells SQL Server that the command text is a procedure name. It does not create or deploy that procedure. The procedure must already exist in the schema and database selected by this connection, and the application's SQL identity must have execution permission.

In `new { Email = email }`, `Email` is the parameter name and `email` is the C# value. In `new { employee.Name }`, C# infers a property named `Name` containing `employee.Name`. Dapper sends values as parameters rather than concatenating them into SQL text.

Every operation's contract

Method and procedure	Dapper call	Required database result
GetByEmail / dbo.GetEmployeeByEmail	QueryFirstOrDefaultAsync<EmployeeEntity>	Zero or more rows; first row or null. Must include correct PasswordHash, Role, Email, and Id for login. Duplicate email rows make login ambiguous.
GetAll / dbo.GetAllEmployees	QueryAsync<EmployeeEntity> then ToList	A result set of employee rows. An empty result set becomes an empty list.
GetById / dbo.GetEmployeeById	QueryFirstOrDefaultAsync<EmployeeEntity>	One employee or no row. The update controller also relies on this result containing the existing hash and role.
Create / dbo.CreateEmployee	QuerySingleAsync<EmployeeEntity>	Exactly one row containing the inserted employee and generated Id. No row or multiple rows cause an exception.
Update / dbo.UpdateEmployee	QueryFirstOrDefaultAsync<EmployeeEntity>	Updated row, or no row if missing. The C# null-coalescing expression turns absence into KeyNotFoundException.
Delete / dbo.DeleteEmployee	ExecuteScalarAsync<int>	The first value of the first row must be a numeric count, normally SELECT @@ROWCOUNT immediately after DELETE.

Why the SQL result matters

Dapper maps matching column names into properties. A result column called Name populates Employee.Name. It does not know that an unrelated column named FullName should populate Name unless the query aliases it appropriately. Missing columns can leave default property values and cause subtle errors.

If GetById omits PasswordHash, the entity defaults that property to an empty string. The current update action then copies that empty string and could overwrite the stored hash. If Role is omitted, the default Employee role could replace a Manager role. Query shape is part of the application contract, even when SQL compiles successfully.

ExecuteScalar reads a selected value; ExecuteAsync reports affected-row counts through a different mechanism. They are not interchangeable. SQL RETURN is a procedure return code, not the SELECT result consumed by ExecuteScalar here. An extra diagnostic SELECT before the intended result can break mapping.

Dapper and EF Core together

Dapper offers direct control over the command and result mapping. EF Core offers model-aware queries and change tracking. The project uses both. Switching a service to Dapper does not remove the need for schema deployment, replace the DbContext in other services, or make SQL Server generate procedures automatically.

Trace: Create sends eight values but no Id; Update sends those values plus Id. Explain why. Answer: SQL generates the identity on insert, while the update must identify the row to change.

8. SQL Stored Procedures Explained

A stored procedure is named SQL code saved inside a SQL Server database. C# calls that name with parameters. SQL Server executes its body and sends result sets or output values back. Storing a procedure in SSMS's editor is not enough: you must execute its CREATE/ALTER batch against the correct database.

Teaching implementations: the following complete examples match the visible EmployeeService parameter and result contracts and the EF snapshot's column names. They were not exported from your SQL Server or executed for this document. Check your actual schema, procedure definitions, constraints, and triggers before using them. The scripts do not add email uniqueness, normalize emails, or define cascade behavior.

Step 1: identify the target database

```
-- Replace the database placeholder before executing in SSMS.
USE [YOUR_DATABASE];
GO
SELECT @@SERVERNAME AS ServerName, DB_NAME() AS DatabaseName;

SELECT TABLE_SCHEMA, TABLE_NAME, COLUMN_NAME,
       DATA_TYPE, IS_NULLABLE, CHARACTER_MAXIMUM_LENGTH
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Employees'
ORDER BY ORDINAL_POSITION;

SELECT SCHEMA_NAME(schema_id) AS SchemaName, name
FROM sys.procedures
WHERE name IN (
    'GetEmployeeByEmail', 'GetAllEmployees', 'GetEmployeeById',
    'CreateEmployee', 'UpdateEmployee', 'DeleteEmployee'
);
```

USE selects the database for this query window. dbo is the schema, a namespace for database objects. Two databases on the same server can have different procedures with the same name. Your application's effective connection string must point to the same server and database that you inspect in SSMS.

go is an SSMS/sqlcmd batch separator, not a SQL statement accepted by SqlCommand. Execute each CREATE OR ALTER PROCEDURE in its own batch. If deploying through migrationBuilder.Sql, omit GO and use separate migrationBuilder.Sql calls.

Step 2: email lookup for login

```
CREATE OR ALTER PROCEDURE [dbo].[GetEmployeeByEmail]
    @Email nvarchar(255)
AS
```

```

BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash,
           Role, Salary, DepartmentId, ClientId
    FROM dbo.Employees
    WHERE Email = @Email;
END;
GO

```

1. CREATE OR ALTER creates the named object or replaces its existing definition.
2. @Email is an input parameter. Its name corresponds to the anonymous Email property in GetByEmail.
3. SET NOCOUNT ON suppresses row-count messages; it does not suppress the SELECT result.
4. The explicit column list makes the returned data contract visible.
5. WHERE compares each row's email with the supplied value. Matching and case sensitivity depend on database collation; no normalization is implemented here.
6. No matching row becomes null in C#, which the login action translates into 401.

No password is supplied to this SQL procedure. It retrieves the saved hash, and ASP.NET Core verifies the supplied password in C#. The query returns all matching rows; QueryFirstOrDefault takes the first. The inspected EF model has no unique Email index, so duplicate prevention remains a separate implementation concern.

Step 3: list employees

```

CREATE OR ALTER PROCEDURE [dbo].[GetAllEmployees]
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash,
           Role, Salary, DepartmentId, ClientId
    FROM dbo.Employees
    ORDER BY Id DESC;
END;
GO

```

This procedure has no input parameters because the current service has no paging or filter arguments. ORDER BY makes the example's ordering explicit. Without an ORDER BY, SQL Server does not promise insertion order. QueryAsync returns the rows and ToList turns them into the interface's required list type.

PasswordHash is returned internally in this teaching example to keep the Employee shape consistent. The entity's JsonIgnore prevents its JSON serialization. A future public list response DTO could select only the fields that clients need and avoid retrieving password hashes for list requests altogether.

Step 4: get one employee by ID

```
CREATE OR ALTER PROCEDURE [dbo].[GetEmployeeById]
    @Id int
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash,
           Role, Salary, DepartmentId, ClientId
    FROM dbo.Employees
    WHERE Id = @Id;
END;
GO
```

Unlike GetAll, this procedure accepts an identifier. With a primary key, the condition can match at most one row. The service also uses this operation before update, so retaining PasswordHash and Role in its result is essential with the current controller.

Step 5: create an employee

```
CREATE OR ALTER PROCEDURE [dbo].[CreateEmployee]
    @Name nvarchar(150),
    @Email nvarchar(255),
    @PhoneNumber nvarchar(10),
    @PasswordHash nvarchar(max),
    @Role nvarchar(50),
    @Salary decimal(18, 2),
    @DepartmentId int,
    @ClientId int = NULL
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Employees
        (Name, Email, PhoneNumber, PasswordHash, Role,
         Salary, DepartmentId, ClientId)
    VALUES
        (@Name, @Email, COALESCE(@PhoneNumber, N''),
         @PasswordHash, @Role, @Salary,
         @DepartmentId, @ClientId);

    DECLARE @NewId int = CONVERT(int, SCOPE_IDENTITY());

    SELECT Id, Name, Email, PhoneNumber, PasswordHash,
           Role, Salary, DepartmentId, ClientId
    FROM dbo.Employees
    WHERE Id = @NewId;
END;
GO
```

1. Each parameter matches a property in `EmployeeService.Create`'s anonymous object. The procedure accepts a hash, not a plaintext password.
2. The parameter lengths mirror the current API profile limits. They are teaching choices, not proof that the deployed table uses the same lengths. Role currently lacks a matching API length/allowlist constraint.
3. `@ClientId` defaults to NULL because a client is optional. `@DepartmentId` is required and should refer to an existing department.
4. The INSERT omits `Id` so SQL Server generates it through the identity column.
5. `COALESCE` substitutes an empty string for a null phone number. This handles the visible mismatch between nullable request `PhoneNumber` and the snapshot's non-null database column.
6. `SCOPE_IDENTITY` returns the most recent identity produced in this scope. `CONVERT` changes its SQL numeric type to the int used by the model.
7. The final SELECT returns the inserted employee. Without it, `QuerySingleAsync` cannot obtain the required result row.

This minimal example assumes ordinary table behavior without custom triggers changing or removing the inserted row. For more complex procedures, design transaction boundaries and result capture deliberately. A stored procedure does not automatically make a multi-statement operation one transaction.

Step 6: update an employee

```
CREATE OR ALTER PROCEDURE [dbo].[UpdateEmployee]
    @Id int,
    @Name nvarchar(150),
    @Email nvarchar(255),
    @PhoneNumber nvarchar(10),
    @PasswordHash nvarchar(max),
    @Role nvarchar(50),
    @Salary decimal(18, 2),
    @DepartmentId int,
    @ClientId int = NULL
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Employees
    SET Name = @Name,
        Email = @Email,
        PhoneNumber = COALESCE(@PhoneNumber, N ''),
        PasswordHash = @PasswordHash,
        Role = @Role,
        Salary = @Salary,
        DepartmentId = @DepartmentId,
        ClientId = @ClientId
    WHERE Id = @Id;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash,
           Role, Salary, DepartmentId, ClientId
```

```
FROM dbo.Employees
WHERE Id = @Id;
END;
GO
```

UPDATE assigns values to the row selected by Id. Omitting WHERE would change every employee, which is why the identifier condition is central to this operation. The final SELECT supplies the result EmployeeService expects; if the employee is absent, it returns no row.

This procedure accepts new Role and PasswordHash values, but the current controller sends the old values. SQL cannot decide that an unused model.Password should have been hashed; responsibility for that mapping is in C#.

Another request can change the row between statements. This example has no optimistic concurrency token or transaction protecting the entire read/update workflow. If concurrency guarantees are required, design the controller contract and SQL procedure together.

Step 7: delete an employee

```
CREATE OR ALTER PROCEDURE [dbo].[DeleteEmployee]
    @Id int
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM dbo.Employees
    WHERE Id = @Id;

    SELECT @@ROWCOUNT AS AffectedRows;
END;
GO
```

DELETE removes the matching row. @@ROWCOUNT gives the affected-row count from the immediately preceding statement. SELECT exposes it as a result value for ExecuteScalarAsync<int>. Do not insert another statement between DELETE and this SELECT, because that can change @@ROWCOUNT.

A selected value of 1 becomes true in the service, and then HTTP 204. A selected 0 becomes false and 404. A foreign-key violation or another SQL exception is neither result; it throws. Decide deliberately how assigned tasks should behave when an employee is deleted.

Step 8: EXEC syntax and safe study data

Use a test database with a real DepartmentId. The following read examples do not modify data:

```
EXEC dbo.GetAllEmployees;
EXEC dbo.GetEmployeeById @Id = 1;
EXEC dbo.GetEmployeeByEmail @Email = N'learner@example.com';
EXEC sys.sp_helptext N'dbo.GetEmployeeByEmail';
```

Use a hash actually produced by PasswordHasher for write examples. Do not put the employee's plaintext password in @PasswordHash. A fabricated string is not a valid login hash.

```
-- Teaching example: replace placeholders before execution.
-- Test writes inside a transaction so you can roll them back.
BEGIN TRANSACTION;

EXEC dbo.CreateEmployee
    @Name = N'Study Employee',
    @Email = N'learner@example.com',
    @PhoneNumber = N'9800000000',
    @PasswordHash = N'PASTE_A_REAL_PASSWORDHASHER_OUTPUT',
    @Role = N'Employee',
    @Salary = 25000.00,
    @DepartmentId = 1,
    @ClientId = NULL;

-- Read the Id from the returned row, then use that Id below.
EXEC dbo.UpdateEmployee
    @Id = 123,
    @Name = N'Study Employee Updated',
    @Email = N'learner@example.com',
    @PhoneNumber = N'9800000000',
    @PasswordHash = N'PASTE_THE_SAME_VALID_HASH',
    @Role = N'Employee',
    @Salary = 28000.00,
    @DepartmentId = 1,
    @ClientId = NULL;

EXEC dbo.DeleteEmployee @Id = 123;

ROLLBACK TRANSACTION;
```

Replace 123 with the returned test Id before running the later statements; execute the example deliberately in stages. Rollback undoes ordinary transactional row changes but does not reclaim identity values. Do not run write experiments against production. Creating through the API is the easier way to obtain a properly hashed password for normal login testing.

Why a C# build cannot verify these procedures

The compiler checks the method call and types, not the SQL text stored on your server. A successful build can coexist with a missing procedure, wrong parameter name, invalid column, or wrong returned result shape. Test both the SQL definition and the API call against the same intended database.

9. Login, Hashes, and JWTs

```
POST /api/auth/login
-> ApiUser credentials match? -> Manager token
-> Otherwise: GetByEmail(username)
-> Employee missing? -> 401
-> Verify stored hash against supplied password
-> Verification fails? -> 401
-> Create JWT with employee.Email, Role, and Id
-> Return { token, expiresAt }
```

Current source: EmployeeApi/Api/v1/AuthApiController.cs. Original line numbers; comments omitted where applicable.

```
1 using EmployeeApi.Data;
2 using EmployeeApi.Model.Employee;
3 using EmployeeApi.ViewModel.Auth;
4 using Microsoft.AspNetCore.Identity;
5 using Microsoft.AspNetCore.Authorization;
6 using Microsoft.AspNetCore.Mvc;
7 using Microsoft.EntityFrameworkCore;
8 using Microsoft.IdentityModel.Tokens;
9 using System.IdentityModel.Tokens.Jwt;
10 using System.Security.Claims;
11 using System.Text;
12 using EmployeeApi.Service.Employee;
13
14 namespace EmployeeApi.Api.v1;
15
16 [ApiController]
17 [Route("api/auth")]
18 public class AuthApiController : ControllerBase
19 {
20     private readonly IConfiguration _configuration;
21     private readonly ApplicationDbContext _context;
22
23     private readonly IEmployeeService _employeeService;
24     public AuthApiController(IConfiguration configuration, ApplicationDbContext context, IEmployeeService
employeeService )
25     {
26         _configuration = configuration;
27         _context = context;
28         _employeeService = employeeService;
29     }
30
31     [AllowAnonymous]
32     [HttpPost("login")]
33     public async Task<ActionResult> Login([FromBody] LoginRequestViewModel request)
34     {
35         var configuredUsername = _configuration["ApiUser:Username"];
36         var configuredPassword = _configuration["ApiUser:Password"];
37
38         if (request.Username == configuredUsername && request.Password == configuredPassword)
39         {
40             var adminToken = CreateToken(request.Username, "Manager");
41             return Ok(adminToken);
42         }
```

```

43
44     var employee = await _employeeService.GetByEmail(request.Username);
45
46     if (employee is null)
47     {
48         return Unauthorized("Invalid username or password.");
49     }
50
51     var hasher = new PasswordHasher<Employee>();
52     var passwordResult = hasher.VerifyHashedPassword(
53         employee,
54         employee.PasswordHash,
55         request.Password);
56
57     if (passwordResult == PasswordVerificationResult.Failed)
58     {
59         return Unauthorized("Invalid username or password.");
60     }
61
62     var token = CreateToken(employee.Email, employee.Role, employee.Id);
63     return Ok(token);
64 }
65
66 private LoginResponseViewModel CreateToken(string username, string role, int? employeeId = null)
67 {
68     var secret = _configuration["Jwt:Secret"]
69         ?? throw new InvalidOperationException("Jwt:Secret is not configured.");
70     var issuer = _configuration["Jwt:Issuer"];
71     var audience = _configuration["Jwt:Audience"];
72     var expirationMinutes = _configuration.GetValue("Jwt:ExpirationMinutes", 30);
73     var expiresAt = DateTime.UtcNow.AddMinutes(expirationMinutes);
74
75     var claims = new List<Claim>
76     {
77         new Claim(JwtRegisteredClaimNames.Sub, username),
78         new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
79         new Claim(ClaimTypes.Name, username),
80         new Claim(ClaimTypes.Role, role)
81     };
82
83     if (employeeId.HasValue)
84     {
85         claims.Add(new Claim(ClaimTypes.NameIdentifier, employeeId.Value.ToString()));
86     }
87
88     var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret));
89     var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
90     var jwt = new JwtSecurityToken(
91         issuer: issuer,
92         audience: audience,
93         claims: claims,
94         expires: expiresAt,
95         signingCredentials: credentials);
96
97     return new LoginResponseViewModel
98     {
99         Token = new JwtSecurityTokenHandler().WriteToken(jwt),
100         ExpiresAt = expiresAt
101     };
102 }
103 }
104

```

Imports and constructor

The Identity import supplies PasswordHasher and verification results. Authorization supplies AllowAnonymous. MVC supplies routing and response helpers. The token, claims, and text imports provide JWT types, claims, signing credentials, and UTF-8 encoding. The constructor stores configuration and employee service dependencies. ApplicationDbContext is still injected and stored, but the active Login implementation accesses employee data through IEmployeeService.

Login method, statement by statement

Code block	Explanation
[AllowAnonymous] [HttpPost("login")]	A user without a token must be able to submit credentials. The route combines with api/auth to make /api/auth/login.
Login([FromBody] LoginRequestViewModel request)	Binds username and password from JSON. Calling this with only an email property does not assign Username.
Read ApiUser:Username and ApiUser:Password	Loads the temporary configured account. This branch is separate from employee database authentication.
Compare both configured values	If they match, the code explicitly calls CreateToken with Manager. The variable name adminToken and username text do not create an Admin role.
GetByEmail(request.Username)	The username value is treated as an email for employee login. It is not looked up against Employee.Name.
employee is null	An absent account returns 401 with the shared invalid-credentials message.
new PasswordHasher<Employee>()	Creates the compatible framework helper used for verification of the saved encoded hash.
VerifyHashedPassword(employee, employee.PasswordHash, request.Password)	Reads parameters from the encoded stored hash and verifies the candidate password. It does not decrypt the hash.
PasswordVerificationResult.Failed	Returns the same 401 message for a failed password check. The current code accepts successful verification but does not persist a replacement hash when SuccessRehashNeeded is returned.
CreateToken(employee.Email, employee.Role, employee.Id)	Uses the database role and ID. The login body cannot directly choose the role.
Ok(token)	Returns the response object with HTTP 200. The frontend decides what page to show afterward.

Password hashing explained

PasswordHasher<Employee> is ASP.NET Core Identity's password helper. Its default IdentityV3 format uses PBKDF2 with a random salt and stores a versioned representation containing the information needed to verify it. The project constructs the helper with defaults; it does not configure its own hashing algorithm or work factor.

The employee object argument is part of the helper's API. The default implementation does not derive the hash from the employee name, ID, or role. The supplied password is the value being hashed. The output is encoded for storage; encoding a password with Base64 alone would not provide equivalent protection. See the [ASP.NET Core PasswordHasher implementation](#).

```
var hasher = new PasswordHasher<EmployeeEntity>();
employee.PasswordHash = hasher.HashPassword(employee, model.Password);

var result = hasher.VerifyHashedPassword(
    employee,
    employee.PasswordHash,
    request.Password);
```

Do not hash the candidate password again and compare strings: fresh salts normally produce different outputs even for the same password. Use VerifyHashedPassword. SQL HASHBYTES produces a different format and is not a substitute for this API. Existing employees with an empty or invalid hash need a proper credential setup/reset flow; adding a database column does not create usable passwords.

CreateToken, statement by statement

1. Read Jwt:Secret. If the configuration entry is absent, throw a configuration error.
2. Read issuer and audience. These identify the expected token issuer and intended recipient. Program validates the same values.
3. Read ExpirationMinutes with a fallback of 30 and compute an expiry from DateTime.UtcNow.
4. Create the claim list. Sub identifies the subject, Jti gives the token a unique ID, Name provides an identity name, and Role carries Manager or Employee (or whatever stored value was passed).
5. If employeeid has a value, add a NameIdentifier claim. The configured ApiUser branch has no employee ID because it did not load an employee record.
6. Convert the secret to UTF-8 bytes and construct a SymmetricSecurityKey.
7. Create SigningCredentials with HmacSha256. Signing lets the receiver detect modification; it does not encrypt the payload.
8. Construct JwtSecurityToken with issuer, audience, claims, expiry, and signing credentials.
9. WriteToken serializes it into the compact bearer-token string.
10. Return Token and ExpiresAt in LoginResponseViewModel.

A JWT contains claims that a holder can decode. Do not put passwords or SMTP secrets in it. Possession of a valid token allows its use until it expires or the server rejects it for another reason, so protect it in transit and

storage.

Roles are captured when a token is issued. Changing a database role or deleting an employee does not automatically revoke an already issued JWT in the current implementation. Program validates the token, but there is no per-request database role refresh or token revocation check.

Working employee login body

```
{  
  "username": "learner@example.com",  
  "password": "StudyOnly123!"  
}
```

This works only after that employee has been created with the matching password hash in the same database. The email label on a frontend input can change without changing the required JSON key username.

10. Manager and Employee Access

Authentication establishes identity from a valid token. Authorization checks whether that identity satisfies an endpoint's requirements. A username containing the word admin has no special effect. Role attributes inspect role claims, not the text of the login name.

```
[Authorize(Roles = "Manager")]  
// Only the Manager role meets this requirement.  
  
[Authorize(Roles = "Manager,Employee")]  
// Either listed role meets this single requirement.
```

A role attribute on the controller applies to its actions. Additional authorization attributes on an action add requirements; they do not simply replace the controller requirement. The comma-separated role list is an OR within that attribute. See [Microsoft's role authorization documentation](#).

Actual endpoint access in the inspected backend

Endpoint	Allowed identity
POST /api/auth/login	Anonymous credential submission
GET /api/employee	Manager or Employee
GET /api/employee/{id}	Manager or Employee
POST /api/employee/new	Manager
PUT /api/employee/{id}	Manager
DELETE /api/employee/{id}	Manager
All /api/department actions	Manager or Employee, including create, update, delete
All /api/client actions	Manager or Employee, including create, update, delete
All /api/assign-task actions	Manager, including GET employee/{employeeId}

If your intended rule is Manager-only Department and Client access, edit the class-level Authorize attribute in those two controller files. If only writes should be Manager-only, keep an appropriate shared read rule and add a Manager requirement to each write action. Decide the access matrix before modifying attributes.

401 versus 403

Situation	Expected result
Wrong login credentials	401 from the login action
Protected endpoint with no valid bearer identity	401 from authentication/authorization handling
Valid Employee token calls a Manager-only action	403: authenticated, but the role requirement is not met
Valid token and role, but requested row is absent	404 from the controller's data lookup

Same UI, different permitted actions

The frontend can show the same application layout to both roles and display different navigation or action buttons. It can read the token's role claim or obtain a user profile from a future endpoint. The backend must still enforce every protected operation, because a user can bypass hidden buttons and call the API with Postman.

Employee-only ownership rules need a separate comparison between the authenticated employee ID and the requested data owner. Merely adding Employee to an action's role list does not add that comparison. The current employee read endpoints expose rows without ownership filtering.

11. The Other Business Modules

These modules follow the same controller-interface-service pattern as employees, but their services use EF Core. Study the shared pattern first: bind a ViewModel, map it to an entity, call the service, then translate its result into an HTTP response.

Department request contracts

Current source: EmployeeApi/ViewModel/Department/CreateDepartmentViewModel.cs. Original line numbers; comments omitted where applicable.

```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace EmployeeApi.ViewModel.Department
4 {
5     public class CreateDepartmentViewModel
6     {
7         [Required]
8         [StringLength(150)]
9         public string Name { get; set; } = string.Empty;
10
11         [StringLength(500)]
12         public string Description { get; set; } = string.Empty;
13     }
14 }
```

Current source: EmployeeApi/ViewModel/Department/UpdateDepartmentViewModel.cs. Original line numbers; comments omitted where applicable.

```
1 namespace EmployeeApi.ViewModel.Department
2 {
3     public class UpdateDepartmentViewModel : CreateDepartmentViewModel
4     {
5     }
6 }
```

Name is required with a 150-character maximum. Description is optional with a 500-character maximum. UpdateDepartmentViewModel inherits the create contract, so it has the same properties and validation without repeating them. This is actual class inheritance, unlike an Employee.Department navigation property.

Department service

Current source: EmployeeApi/Service/Department/IDepartmentService.cs. Original line numbers; comments omitted where applicable.

```
1 using DepartmentEntity = EmployeeApi.Model.Department.Department;
2
3 namespace EmployeeApi.Service.Department
4 {
5     public interface IDepartmentService
```

```

6    {
7        Task<List<DepartmentEntity>> GetAll();
8
9        Task<DepartmentEntity?> GetById(int id);
10
11       Task<DepartmentEntity> Create(DepartmentEntity department);
12
13       Task<DepartmentEntity> Update(DepartmentEntity department);
14
15       Task<bool> Delete(int id);
16    }
17 }

```

Current source: EmployeeApi/Service/Department/DepartmentService.cs. Original line numbers; comments omitted where applicable.

```

1  using EmployeeApi.Data;
2  using Microsoft.EntityFrameworkCore;
3  using DepartmentEntity = EmployeeApi.Model.Department.Department;
4
5  namespace EmployeeApi.Service.Department
6  {
7      public class DepartmentService : IDepartmentService
8      {
9          private readonly ApplicationDbContext _context;
10
11         public DepartmentService(ApplicationDbContext context)
12         {
13             _context = context;
14         }
15
16         public async Task<List<DepartmentEntity>> GetAll()
17         {
18             return await _context.Departments.ToListAsync();
19         }
20
21         public async Task<DepartmentEntity?> GetById(int id)
22         {
23             return await _context.Departments.FindAsync(id);
24         }
25
26         public async Task<DepartmentEntity> Create(DepartmentEntity department)
27         {
28             _context.Departments.Add(department);
29
30             await _context.SaveChangesAsync();
31
32             return department;
33         }
34
35         public async Task<DepartmentEntity> Update(DepartmentEntity department)
36         {
37             var existingDepartment =
38                 await _context.Departments.FindAsync(department.Id)
39                 ?? throw new KeyNotFoundException(
40                     $"Department {department.Id} was not found.");
41
42             _context.Entry(existingDepartment)
43                 .CurrentValues
44                 .SetValues(department);

```

```

45
46         await _context.SaveChangesAsync();
47
48         return existingDepartment;
49     }
50
51     public async Task<bool> Delete(int id)
52     {
53         var department =
54             await _context.Departments.FindAsync(id);
55
56         if (department is null)
57         {
58             return false;
59         }
60
61         _context.Departments.Remove(department);
62
63         await _context.SaveChangesAsync();
64
65         return true;
66     }
67 }
68 }

```

1. The constructor stores the DI-provided ApplicationDbContext.
2. GetAll uses ToListAsync to execute a SQL query and materialize department objects.
3. GetByld uses FindAsync with the primary key. EF can return an already tracked matching object or query the database.
4. Create calls Add to mark a new entity as Added; SaveChangesAsync sends the INSERT. Add alone does not commit it.
5. Update finds the tracked entity and throws if missing. CurrentValues.SetValues copies scalar values from the incoming entity into the tracked one; SaveChangesAsync persists the changes.
6. Delete finds the entity, returns false when absent, otherwise marks it Deleted and saves. It returns true after persistence.

Department controller

Current source: EmployeeApi/Api/v1/DepartmentApiController.cs. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Service.Department;
2 using EmployeeApi.ViewModel.Department;
3 using Microsoft.AspNetCore.Authorization;
4 using Microsoft.AspNetCore.Mvc;
5 using DepartmentEntity = EmployeeApi.Model.Department.Department;
6
7 namespace EmployeeApi.Api.v1
8 {
9     [ApiController]
10    [Authorize(Roles = "Manager,Employee")]
11    [Route("api/department")]
12    public class DepartmentApiController : ControllerBase
13    {

```

```

14     private readonly IDepartmentService _service;
15
16     public DepartmentApiController(
17         IDepartmentService service)
18     {
19         _service = service;
20     }
21
22     [HttpGet]
23     public async Task<IActionResult> Get()
24     {
25         return Ok(await _service.GetAll());
26     }
27
28     [HttpGet("{id:int}")]
29     public async Task<IActionResult> GetById(int id)
30     {
31         var department =
32             await _service.GetById(id);
33
34         return department is null
35             ? NotFound()
36             : Ok(department);
37     }
38
39     [HttpPost]
40     public async Task<IActionResult> Create(
41         [FromBody] CreateDepartmentViewModel model)
42     {
43         var department = new DepartmentEntity
44         {
45             Name = model.Name,
46             Description = model.Description
47         };
48
49         var createdDepartment =
50             await _service.Create(department);
51
52         return CreatedAtAction(
53             nameof(GetById),
54             new { id = createdDepartment.Id },
55             createdDepartment);
56     }
57
58     [HttpPut("{id:int}")]
59     public async Task<IActionResult> Update(
60         int id,
61         [FromBody] UpdateDepartmentViewModel model)
62     {
63         if (await _service.GetById(id) is null)
64         {
65             return NotFound();
66         }
67
68         var department = new DepartmentEntity
69         {
70             Id = id,
71             Name = model.Name,
72             Description = model.Description
73         };
74
75         return Ok(

```

```

76         await _service.Update(department));
77     }
78
79     [HttpDelete("{id:int}")]
80     public async Task<IActionResult> Delete(int id)
81     {
82         return await _service.Delete(id)
83             ? NoContent()
84             : NotFound();
85     }
86 }
87 }

```

GET returns the service list. GET by ID maps null to 404. POST maps Name and Description and returns CreatedAtAction. PUT first checks existence, builds an entity using the route ID, and returns the update result. DELETE translates bool into 204 or 404. The class-level Manager,Employee attribute applies to every one of these actions.

The Department relationship is configured as required for employees and the snapshot includes cascade deletion. Before using department DELETE, inspect the actual database foreign key and decide whether deleting related employees is intended.

Client request contracts

Current source: EmployeeApi/ViewModel/Client/CreateClientViewModel.cs. Original line numbers; comments omitted where applicable.

```

1 using System.ComponentModel.DataAnnotations;
2
3 namespace EmployeeApi.ViewModel.Client
4 {
5     public class CreateClientViewModel
6     {
7         [Required]
8         [StringLength(100)]
9         public string ClientName { get; set; } = string.Empty;
10        [Required]
11        [StringLength(10)]
12        [Phone]
13        public string PhoneNumber { get; set; } = string.Empty;
14        [Required]
15        [StringLength(100)]
16        public string ProjectName { get; set; } = string.Empty;
17    }
18 }
19

```

Current source: EmployeeApi/ViewModel/Client/UpdateClientViewModel.cs. Original line numbers; comments omitted where applicable.

```

1 namespace EmployeeApi.ViewModel.Client
2 {
3     public class UpdateClientViewModel:CreateClientViewModel
4     {
5     }

```

```
6 }  
7
```

ClientName and ProjectName are required with 100-character maximums. PhoneNumber is required, capped at 10 characters, and validated using Phone. Update inherits the create contract, so all these fields remain required.

Client service

Current source: EmployeeApi/Service/Client/IClientService.cs. Original line numbers; comments omitted where applicable.

```
1 using ClientEntity = EmployeeApi.Model.Client.Client;  
2 namespace EmployeeApi.Service.Client  
3 {  
4     public interface IClientService  
5     {  
6         Task<List<ClientEntity>> GetAll();  
7  
8         Task<ClientEntity?> GetById(int id);  
9  
10        Task<ClientEntity> Create(ClientEntity client);  
11  
12        Task<ClientEntity> Update(int id, ClientEntity client);  
13  
14        Task<bool> Delete(int id);  
15    }  
16 }  
17
```

Current source: EmployeeApi/Service/Client/ClientService.cs. Original line numbers; comments omitted where applicable.

```
1 using EmployeeApi.Data;  
2 using Microsoft.EntityFrameworkCore;  
3 using ClientEntity = EmployeeApi.Model.Client.Client;  
4 namespace EmployeeApi.Service.Client  
5 {  
6     public class ClientService : IClientService  
7     {  
8         private readonly ApplicationDbContext _context;  
9         public ClientService(ApplicationDbContext context)  
10        {  
11            _context = context;  
12        }  
13  
14        public async Task<List<ClientEntity>> GetAll()  
15        {  
16            return await _context.Clients.ToListAsync();  
17        }  
18  
19        public async Task<ClientEntity?> GetById(int id)  
20        {  
21            return await _context.Clients.FindAsync(id);  
22        }  
23  
24        public async Task<ClientEntity> Create(ClientEntity client)  
25        {  
26            _context.Clients.Add(client);  
27        }  
28    }  
29 }
```

```

27         await _context.SaveChangesAsync();
28         return client;
29     }
30
31     public async Task<ClientEntity> Update(int id, ClientEntity client)
32     {
33         var existingClient = await _context.Clients.FindAsync(id);
34         if (existingClient == null)
35             throw new InvalidOperationException("Client not found");
36
37         _context.Entry(existingClient).CurrentValues.SetValues(client);
38         await _context.SaveChangesAsync();
39         return existingClient;
40     }
41
42     public async Task<bool> Delete(int id)
43     {
44         var client = await _context.Clients.FindAsync(id);
45         if (client == null)
46             return false;
47
48         _context.Clients.Remove(client);
49         await _context.SaveChangesAsync();
50         return true;
51     }
52 }
53 }
54

```

The CRUD behavior is similar to `DepartmentService`. A small contract difference matters: `ClientService.Update` takes both `id` and `client`. It finds by the `id` argument and throws `InvalidOperationException` if missing. `SetValues` copies values from `client` into the tracked row, so callers should keep `client.Id` consistent with `id`. The current controller does this.

Client controller

Current source: `EmployeeApi/Api/v1/ClientApiController.cs`. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Service.Client;
2 using EmployeeApi.ViewModel.Client;
3 using Microsoft.AspNetCore.Authorization;
4 using Microsoft.AspNetCore.Mvc;
5 using ClientEntity= EmployeeApi.Model.Client.Client;
6
7 namespace EmployeeApi.Api.v1
8 {
9     [ApiController]
10
11     [Route("api/client")]
12     [Authorize(Roles = "Manager,Employee")]
13     public class ClientApiController : ControllerBase
14     {
15         private readonly IClientService _service;
16         public ClientApiController(IClientService clientService)
17         {
18             _service = clientService;
19         }
20         [HttpGet]

```

```

21     public async Task<IActionResult> Get()
22     {
23         return Ok(await _service.GetAll());
24     }
25
26     [HttpGet("{id:int}")]
27     public async Task<IActionResult> GetById(int id)
28     {
29         var client = await _service.GetById(id);
30         return client is null ? NotFound() : Ok(client);
31     }
32
33     [HttpPost]
34     public async Task<IActionResult> Create([FromBody] CreateClientViewModel model)
35     {
36         var client = new ClientEntity
37         {
38             ClientName = model.ClientName,
39             PhoneNumber = model.PhoneNumber,
40             ProjectName = model.ProjectName,
41         };
42         var createdClient = await _service.Create(client);
43         return CreatedAtAction(nameof(GetById), new { id = createdClient.Id }, createdClient);
44     }
45
46     [HttpPut("{id:int}")]
47     public async Task<IActionResult> Update(int id, [FromBody] UpdateClientViewModel model)
48     {
49         if (await _service.GetById(id) is null)
50         {
51             return NotFound();
52         }
53         var client = new ClientEntity
54         {
55             Id = id,
56             ClientName = model.ClientName,
57             PhoneNumber = model.PhoneNumber,
58             ProjectName = model.ProjectName,
59         };
60         var updatedClient = await _service.Update(id, client);
61         return Ok(updatedClient);
62     }
63     [HttpDelete("{id:int}")]
64     public async Task<IActionResult> Delete(int id)
65     {
66         if (await _service.GetById(id) is null)
67         {
68             return NotFound();
69         }
70         await _service.Delete(id);
71         return NoContent();
72     }
73 }
74 }
75

```

Create maps three request properties and returns 201. Update checks existence and maps Id from the URL. Delete performs its own existence check, calls the bool-returning service, then returns 204 without inspecting that bool. If another request deletes the client between the check and the service call, the response can still be 204.

The optional employee-to-client relationship does not by itself define a user-friendly response for deleting a referenced client. SQL can reject the operation depending on the actual foreign key. The controller does not catch that database exception.

Assigned Task request contracts

Current source: EmployeeApi/ViewModel/AssignTask/CreateAssignTaskViewModel.cs. Original line numbers; comments omitted where applicable.

```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace EmployeeApi.ViewModel.AssignTask;
4
5 public class CreateAssignTaskViewModel
6 {
7     [Range(1, int.MaxValue)]
8     public int EmployeeId { get; set; }
9
10    [Required]
11    [StringLength(200)]
12    public string Title { get; set; } = string.Empty;
13
14    [StringLength(2000)]
15    public string Description { get; set; } = string.Empty;
16
17    public DateTime? DueDate { get; set; }
18
19    [Required]
20    [StringLength(50)]
21    public string Status { get; set; } = "Pending";
22 }
23
```

Current source: EmployeeApi/ViewModel/AssignTask/UpdateAssignTaskViewModel.cs. Original line numbers; comments omitted where applicable.

```
1 namespace EmployeeApi.ViewModel.AssignTask;
2
3 public class UpdateAssignTaskViewModel : CreateAssignTaskViewModel
4 {
5 }
6
```

EmployeeId must be positive, Title is required and limited to 200 characters, Description is limited to 2000, DueDate is nullable, and Status is required with a maximum of 50. The current contract checks length but does not limit Status to a defined set such as Pending and Completed. Update inherits the same fields.

Assigned Task service

Current source: EmployeeApi/Service/AssignTask/IAssignTaskService.cs. Original line numbers; comments omitted where applicable.

```
1 using AssignedTaskEntity = EmployeeApi.Model.AssignTask.AssignedTask;
2
```

```

3 namespace EmployeeApi.Service.AssignTask;
4
5 public interface IAssignTaskService
6 {
7     Task<List<AssignedTaskEntity>> GetAll();
8
9     Task<List<AssignedTaskEntity>> GetByEmployeeId(int employeeId);
10
11     Task<AssignedTaskEntity?> GetById(int id);
12
13     Task<AssignedTaskEntity> Create(AssignedTaskEntity assignedTask);
14
15     Task<AssignedTaskEntity> Update(AssignedTaskEntity assignedTask);
16
17     Task<bool> Delete(int id);
18 }
19

```

Current source: EmployeeApi/Service/AssignTask/AssignTaskService.cs. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Data;
2 using Microsoft.EntityFrameworkCore;
3 using AssignedTaskEntity = EmployeeApi.Model.AssignTask.AssignedTask;
4
5 namespace EmployeeApi.Service.AssignTask;
6
7 public class AssignTaskService : IAssignTaskService
8 {
9     private readonly ApplicationDbContext _context;
10
11     public AssignTaskService(ApplicationDbContext context)
12     {
13         _context = context;
14     }
15
16     public async Task<List<AssignedTaskEntity>> GetAll()
17     {
18         return await _context.AssignedTasks
19             .OrderByDescending(task => task.AssignedOn)
20             .ToListAsync();
21     }
22
23     public async Task<List<AssignedTaskEntity>> GetByEmployeeId(int employeeId)
24     {
25         return await _context.AssignedTasks
26             .Where(task => task.EmployeeId == employeeId)
27             .OrderByDescending(task => task.AssignedOn)
28             .ToListAsync();
29     }
30
31     public async Task<AssignedTaskEntity?> GetById(int id)
32     {
33         return await _context.AssignedTasks.FindAsync(id);
34     }
35
36     public async Task<AssignedTaskEntity> Create(AssignedTaskEntity assignedTask)
37     {
38         await EnsureEmployeeExists(assignedTask.EmployeeId);
39     }
40

```

```

40     _context.AssignedTasks.Add(assignedTask);
41     await _context.SaveChangesAsync();
42     return assignedTask;
43 }
44
45 public async Task<AssignedTaskEntity> Update(AssignedTaskEntity assignedTask)
46 {
47     await EnsureEmployeeExists(assignedTask.EmployeeId);
48
49     var existingTask = await _context.AssignedTasks.FindAsync(assignedTask.Id)
50         ?? throw new KeyNotFoundException($"Assigned task {assignedTask.Id} was not found.");
51
52     assignedTask.AssignedOn = existingTask.AssignedOn;
53     _context.Entry(existingTask).CurrentValues.SetValues(assignedTask);
54     await _context.SaveChangesAsync();
55     return existingTask;
56 }
57
58 public async Task<bool> Delete(int id)
59 {
60     var assignedTask = await _context.AssignedTasks.FindAsync(id);
61
62     if (assignedTask is null)
63     {
64         return false;
65     }
66
67     _context.AssignedTasks.Remove(assignedTask);
68     await _context.SaveChangesAsync();
69     return true;
70 }
71
72 private async Task EnsureEmployeeExists(int employeeId)
73 {
74     if (!await _context.Employees.AnyAsync(employee => employee.Id == employeeId))
75     {
76         throw new KeyNotFoundException($"Employee {employeeId} was not found.");
77     }
78 }
79 }
80

```

1. GetAll orders assignments by AssignedOn descending, so the newest appear first.
2. GetByEmployeeId adds a Where filter on EmployeeId before ordering. This filters the requested employee, not automatically the logged-in employee.
3. GetById uses FindAsync.
4. Create awaits EnsureEmployeeExists before adding and saving a task.
5. Update validates the target employee, finds the task, and copies the existing AssignedOn into the incoming object so editing does not reset the original assignment timestamp.
6. SetValues updates the tracked task, then SaveChangesAsync writes it.
7. Delete returns false for absence or saves a deletion and returns true.
8. EnsureEmployeeExists uses AnyAsync to ask whether a matching employee exists. It does not need to load the entire employee entity.

Assigned Task controller

Current source: EmployeeApi/Api/v1/AssignTaskApiController.cs. Original line numbers; comments omitted where applicable.

```
1 using EmployeeApi.Service.AssignTask;
2 using EmployeeApi.ViewModel.AssignTask;
3 using Microsoft.AspNetCore.Authorization;
4 using Microsoft.AspNetCore.Mvc;
5 using AssignedTaskEntity = EmployeeApi.Model.AssignTask.AssignedTask;
6
7 namespace EmployeeApi.Api.v1;
8
9 [ApiController]
10 [Route("api/assign-task")]
11 [Authorize(Roles = "Manager")]
12 public class AssignTaskApiController : ControllerBase
13 {
14     private readonly IAssignTaskService _service;
15
16     public AssignTaskApiController(IAssignTaskService service)
17     {
18         _service = service;
19     }
20
21     [HttpGet]
22     public async Task<IActionResult> Get()
23     {
24         return Ok(await _service.GetAll());
25     }
26
27     [HttpGet("{id:int}")]
28     public async Task<IActionResult> GetById(int id)
29     {
30         var assignedTask = await _service.GetById(id);
31         return assignedTask is null ? NotFound() : Ok(assignedTask);
32     }
33
34     [HttpGet("employee/{employeeId:int}")]
35     public async Task<IActionResult> GetByEmployeeId(int employeeId)
36     {
37         return Ok(await _service.GetByEmployeeId(employeeId));
38     }
39
40     [HttpPost]
41     public async Task<IActionResult> Create([FromBody] CreateAssignTaskViewModel model)
42     {
43         try
44         {
45             var assignedTask = Map(model);
46             var createdTask = await _service.Create(assignedTask);
47             return CreatedAtAction(nameof(GetById), new { id = createdTask.Id }, createdTask);
48         }
49         catch (KeyNotFoundException exception)
50         {
51             return NotFound(exception.Message);
52         }
53     }
54
55     [HttpPut("{id:int}")]
56     public async Task<IActionResult> Update(int id, [FromBody] UpdateAssignTaskViewModel model)
57     {
```

```

58     try
59     {
60         var assignedTask = Map(model);
61         assignedTask.Id = id;
62         return Ok(await _service.Update(assignedTask));
63     }
64     catch (KeyNotFoundException exception)
65     {
66         return NotFound(exception.Message);
67     }
68 }
69
70 [HttpDelete("{id:int}")]
71 public async Task<IActionResult> Delete(int id)
72 {
73     return await _service.Delete(id) ? NoContent() : NotFound();
74 }
75
76 private static AssignedTaskEntity Map(CreateAssignTaskViewModel model)
77 {
78     return new AssignedTaskEntity
79     {
80         EmployeeId = model.EmployeeId,
81         Title = model.Title,
82         Description = model.Description,
83         DueDate = model.DueDate,
84         Status = model.Status
85     };
86 }
87 }
88

```

The private static Map helper converts a CreateAssignTaskViewModel to an AssignedTask entity. UpdateAssignTaskViewModel inherits that create type, so the same helper accepts it. Static indicates that Map does not depend on an instance field.

Create and Update catch KeyNotFoundException and return 404 with its message. That covers missing employees/tasks reported by the service; it is not a catch-all for database problems. GetByEmployeeId returns a list, including an empty list when there are no assignments. The class-level Manager rule also protects that route, so an Employee cannot currently use it to view their tasks.

Study task: compare EmployeeService.Update and DepartmentService.Update. The employee version passes parameters to a procedure and maps its result. The department version changes a tracked entity and calls SaveChangesAsync. Both satisfy their service contract, but their persistence mechanisms differ.

12. Employee Welcome Email

This project's email implementation uses MailSettings and IEmployeeEmailSender. The YangOneAppConfig, SMTMConfig, yoconfig.json, and IEmailSender names from the earlier reference project are not the names used by this backend.

```
appsettings / environment variables
-> MailSettings section
-> Program.Configure<MailSettings>
-> IOptions<MailSettings>
-> SmtplibEmployeeEmailSender constructor -> options.Value

EmployeeApiController.Create
-> EmployeeService.Create -> database row saved
-> IEmployeeEmailSender.SendAccountCreatedAsync
-> MailMessage + SmtplibClient
-> SMTP provider -> recipient mailbox
```

Step 1: the typed settings class

Current source: EmployeeApi/Configuration/MailSettings.cs. Original line numbers; comments omitted where applicable.

```
1 namespace EmployeeApi.Configuration;
2
3 public class MailSettings
4 {
5     public bool Enabled { get; set; }
6
7     public string Host { get; set; } = string.Empty;
8
9     public int Port { get; set; }
10
11     public bool UseSsl { get; set; }
12
13     public string UserName { get; set; } = string.Empty;
14
15     public string Password { get; set; } = string.Empty;
16
17     public string FromEmail { get; set; } = string.Empty;
18
19     public string FromName { get; set; } = string.Empty;
20 }
21
```

Setting	Purpose
Enabled	Feature switch. False makes the sender return immediately without constructing or sending a message.
Host	SMTP server hostname, such as smtp.gmail.com.

Setting	Purpose
Port	Server connection port. It must match the transport mode supported by this client and provider.
UseSsl	Assigned to <code>SmtpClient.EnableSsl</code> to request STARTTLS encryption.
UserName and Password	Provider authentication credentials. These are unrelated to an employee's application login password.
FromEmail and FromName	Sender address and display name placed in the email. The provider may require a verified or authorized sender address.

Binding matches configuration keys to properties. The class does not read JSON by itself. Program supplies the section through `Configure<MailSettings>`. Missing values fall back to these defaults; there is no `ValidateOnStart` validation in the current setup.

Step 2: the sender contract

Current source: `EmployeeApi/Service/Notifications/IEmployeeEmailSender.cs`. Original line numbers; comments omitted where applicable.

```

1 using EmployeeEntity = EmployeeApi.Model.Employee.Employee;
2
3 namespace EmployeeApi.Service.Notifications;
4
5 public interface IEmployeeEmailSender
6 {
7     Task SendAccountCreatedAsync(EmployeeEntity employee, string plainPassword);
8 }
9

```

The interface declares one asynchronous operation. The `Employee` supplies the name, email, and role. The separate `plainPassword` argument supplies the password received during account creation, because a hash cannot be reversed to recover it. `Task` indicates completion or failure, with no separate success value returned.

Step 3: the SMTP implementation

Current source: `EmployeeApi/Service/Notifications/SmtpEmployeeEmailSender.cs`. Original line numbers; comments omitted where applicable.

```

1 using EmployeeApi.Configuration;
2 using Microsoft.Extensions.Options;
3 using System.Net;
4 using System.Net.Mail;
5 using EmployeeEntity = EmployeeApi.Model.Employee.Employee;
6
7 namespace EmployeeApi.Service.Notifications;
8

```

```

8
9 public class SmtplibEmployeeEmailSender : IEmployeeEmailSender
10 {
11     private readonly MailSettings _settings;
12
13     public SmtplibEmployeeEmailSender(IOptions<MailSettings> options)
14     {
15         _settings = options.Value;
16     }
17
18     public async Task SendAccountCreatedAsync(EmployeeEntity employee, string plainPassword)
19     {
20         if (!_settings.Enabled)
21         {
22             return;
23         }
24
25         using var message = new MailMessage
26         {
27             From = new MailAddress(_settings.FromEmail, _settings.FromName),
28             Subject = "Your employee account has been created",
29             Body = $"
30 Hello {employee.Name},
31
32 Your employee account has been created.
33
34 Login Email: {employee.Email}
35 Password: {plainPassword}
36 Role: {employee.Role}
37
38 Please change your password after login.
39 """,
40             IsBodyHtml = false
41         };
42
43         message.To.Add(employee.Email);
44
45         using var smtpClient = new SmtpClient(_settings.Host, _settings.Port)
46         {
47             EnableSsl = _settings.UseSsl,
48             Credentials = new NetworkCredential(_settings.UserName, _settings.Password)
49         };
50
51         await smtpClient.SendMailAsync(message);
52     }
53 }
54

```

1. The configuration import gives access to MailSettings; the options import supplies IOption; System.Net supplies NetworkCredential; System.Net.Mail supplies MailMessage, MailAddress, and SmtpClient.
2. The class implements IEmployeeEmailSender, allowing it to be substituted wherever that interface is requested.
3. The constructor receives IOption<MailSettings> and saves options.Value, the bound settings object.
4. The Enabled check exits without sending when email is disabled. The caller receives a completed Task; it is not told that delivery was skipped.

5. using var message ensures MailMessage is disposed at method exit.
6. From constructs the sender address and display name. Subject supplies the subject line.
7. The interpolated raw string body inserts employee.Name, employee.Email, plainPassword, and employee.Role into readable text. Raw string syntax preserves a multi-line body without repeated newline escape sequences.
8. IsBodyHtml=false means the body is plain text. HTML tags would not be interpreted as markup.
9. message.To.Add(employee.Email) sets the recipient from the created employee.
10. SmtpClient uses the configured host and port; EnableSsl takes UseSsl; Credentials receives the provider username and password.
11. SendMailAsync submits the message to the SMTP server. Exceptions propagate because this method has no catch.
12. The using declarations dispose both network/message objects afterward.

Successful SMTP submission does not prove the email reached the inbox. The provider may accept it and later bounce or filter it. The code does not track delivery, queue retries, or expose a resend endpoint.

Step 4: registration and use

```
// Program.cs
builder.Services.Configure<MailSettings>(
    builder.Configuration.GetSection("MailSettings"));

// EmployeeApiServiceRegistrar.cs
services.AddScoped<IEmployeeEmailSender, SmtplibEmployeeEmailSender>();

// EmployeeApiController constructor
public EmployeeApiController(
    IEmployeeService service,
    IEmployeeEmailSender employeeEmailSender)
{
    _service = service;
    _employeeEmailSender = employeeEmailSender;
}

// Create action, after the employee is saved
await _employeeEmailSender.SendAccountCreatedAsync(
    createdEmployee, model.Password);
```

Another service can ask for IEmployeeEmailSender in its constructor and call the same method when it has the necessary data. It should not instantiate the concrete SMTP sender and manually reread appsettings. For a different kind of email, add an appropriate method or separate contract rather than using an account-created message with misleading content.

Step 5: configure a provider

1. Obtain the SMTP hostname, STARTTLS port, authentication username, app password or SMTP credential, and permitted From address.
2. Set the MailSettings values in configuration. Start with Enabled=false while testing unrelated database behavior.
3. For Gmail, use smtp.gmail.com and STARTTLS port 587 with UseSsl=true. Use the full sender Gmail address and an app password when the account permits app passwords.
4. Gmail app passwords require 2-Step Verification and may be unavailable for certain managed accounts or security settings. Follow the account/provider instructions rather than using the ordinary account password.
5. Store the SMTP credential in an environment variable or secret store, for example MailSettings_Password. Do not put it in the PDF, logs, or source control.
6. Set Enabled=true and restart the app so the current IOption-based sender uses the intended configuration.
7. Use a test employee with an inbox you control and trigger employee creation. Inspect both the API result and mailbox/provider delivery logs.

Provider values are documented by [Google's Gmail SMTP settings](#); app-password eligibility and setup are in [Google's app password guide](#).

Port compatibility: this System.Net.Mail.SmtpClient implementation uses STARTTLS through EnableSsl. Implicit TLS on port 465 is a different connection mode and is not supported by that property. Do not switch this code to 465 just because a provider lists it as an SSL port. See [SmtpClient.EnableSsl](#).

Current credential-email limitation

The implemented message includes a plaintext application password and says to change it after login. There is currently no self-service change-password endpoint in these controllers, and the Manager update action ignores its optional password field. The email's instruction does not create that capability.

For a future production onboarding design, prefer a short-lived one-time account setup link so the employee chooses their password. This would require a token lifecycle and corresponding API/UI work. The current implementation is documented here as it exists; it does not provide that flow.

13. Migrations and Deployment

A migration is versioned code describing database changes. Creating a migration writes a file; applying it changes a database. Saving an Employee or calling a stored procedure is an ordinary data operation, not a migration.

The current startup initializer

Current source: EmployeeApi/Data/EmployeeApiDatabaseInitializer.cs. Original line numbers; comments omitted where applicable.

```
1 using Microsoft.EntityFrameworkCore;
2
3 namespace EmployeeApi.Data;
4
5 public static class EmployeeApiDatabaseInitializer
6 {
7     public static async Task EnsureSchemaAsync(ApplicationDbContext dbContext)
8     {
9         await dbContext.Database.MigrateAsync();
10    }
11 }
12
```

Program creates a scope, resolves ApplicationDbContext, and awaits this method. MigrateAsync applies pending migrations to the configured database. Consequently, simply starting this backend can change the schema. If the database is unavailable or the SQL identity lacks migration permissions, startup can fail before requests are served.

A real schema-change migration

Current source: EmployeeApi/Migrations/20260907055854_addrolebaseauth.cs. Original line numbers; comments omitted where applicable.

```
1 using Microsoft.EntityFrameworkCore.Migrations;
2
3 #nullable disable
4
5 namespace EmployeeApi.Migrations
6 {
7     public partial class addrolebaseauth : Migration
8     {
9         protected override void Up(MigrationBuilder migrationBuilder)
10        {
11            migrationBuilder.AddColumn<string>(
12                name: "PasswordHash",
13                table: "Employees",
14                type: "nvarchar(max)",
15                nullable: false,
16                defaultValue: "");
17        }
18    }
19
```

```

20         migrationBuilder.AddColumn<string>(
21             name: "Role",
22             table: "Employees",
23             type: "nvarchar(max)",
24             nullable: false,
25             defaultValue: "Employee");
26     }
27
28     protected override void Down(MigrationBuilder migrationBuilder)
29     {
30         migrationBuilder.DropColumn(
31             name: "PasswordHash",
32             table: "Employees");
33
34         migrationBuilder.DropColumn(
35             name: "Role",
36             table: "Employees");
37     }
38 }
39 }
40 }
41

```

Up adds PasswordHash and Role columns to Employees. Both are non-null. The migration supplies an empty string for PasswordHash and Employee for Role when the columns are introduced. These values do not create a valid password for pre-existing employees. Down removes both columns, which would remove the corresponding stored data.

The Role initializer in Employee.cs is a C# default. The defaultValue in a migration is SQL schema/data behavior. They are separate mechanisms. Editing an already applied migration does not cause EF to rerun it against a database whose migration history already records it.

An empty migration does not deploy a procedure

Current source: EmployeeApi/Migrations/20260907091727_addspeg.cs. Original line numbers; comments omitted where applicable.

```

1 using Microsoft.EntityFrameworkCore.Migrations;
2
3 #nullable disable
4
5 namespace EmployeeApi.Migrations
6 {
7     public partial class addspeg : Migration
8     {
9         protected override void Up(MigrationBuilder migrationBuilder)
10        {
11        }
12
13        protected override void Down(MigrationBuilder migrationBuilder)
14        {
15        }
16    }
17 }
18
19

```

This migration's Up and Down are empty. Its name does not cause SQL Server to create a stored procedure. EF does not inspect Dapper command strings and generate matching SQL bodies.

Existing migration history in the repository

Migration filename prefix	Visible operation
20260618064851_test	Conditional SQL creates/adjusts Departments, Employees, and AssignedTasks, plus the employee department index and cascading foreign key.
20260904055621_Addempphnum	Empty Up and Down.
20260904060241_Addempphnumsm	Adds Employees.PhoneNumber.
20260904070738_Client	Creates Clients.
20260904073123_Clrnttoemp	Adds nullable Employee.ClientId, its index, and the client foreign key.
20260907055854_addrolebaseauth	Adds PasswordHash and Role.
20260907062727_addrolebaseauths	Empty Up and Down.
20260907091727_addspeg	Empty Up and Down; no procedure deployment.

The Designer files describe the target EF model for each migration. ApplicationDbContextModelSnapshot records the latest model shape for future comparisons. Neither is a live export of your database. Do not hand-edit the snapshot to pretend a missing schema change was applied.

Do I need a migration?

Change	Required database work
Add a persisted property, table, index, or relationship	Normally create and review an EF migration, then apply it through your deployment process.
Create/alter a procedure for a Dapper call	Deploy the SQL definition. This can be a versioned SQL script or explicit migrationBuilder.Sql in a migration.
Switch Create/Update from EF to Dapper with the same schema	No automatic entity-schema migration is needed, but the called procedures must exist and match the contract.
Add Authorize attributes or change JWT claims	No schema migration unless the change also introduces persisted data.
Change SMTP settings or email text	No migration for the current email implementation.

Change	Required database work
Add a request-only ViewModel property	No schema change by itself. Map it to behavior deliberately.
Create/update an employee row	Ordinary data write. Do not generate a migration for each employee.
Create this PDF	No migration.

Commands to understand before using them

```
# Run from the solution root. These are study commands, not executed here.
dotnet ef migrations list --project EmployeeApi
dotnet ef migrations has-pending-model-changes --project EmployeeApi

# Only when an intentional model/schema change needs a new migration:
dotnet ef migrations add MeaningfulChangeName --project EmployeeApi

# Review the generated Up/Down methods before applying:
dotnet ef database update --project EmployeeApi

# Produce a reviewable SQL migration script:
dotnet ef migrations script --idempotent --project EmployeeApi
```

The dotnet-ef CLI tool must be available with a version compatible with the project. Adding EF Tools in a project does not necessarily install the global CLI command. Some commands construct the application's design-time services, so use the correct configuration and development environment.

For procedure deployment through EF, add explicit SQL to a reviewed new migration's Up method, with one procedure definition per `migrationBuilder.Sql` call and no GO. Its Down behavior should restore the previous definition when replacing a procedure, or drop a newly introduced procedure when that rollback is appropriate. See [EF Core's migration management guide](#).

Read-only verification in SSMS

```
SELECT MigrationId, ProductVersion
FROM dbo.__EFMigrationsHistory
ORDER BY MigrationId;

SELECT Id, Name, Email, Role,
       CASE WHEN PasswordHash IS NULL OR PasswordHash = N''
            THEN N'Missing' ELSE N'Present' END AS HashState
FROM dbo.Employees;

SELECT OBJECT_ID(N'dbo.GetEmployeeByEmail', N'P') AS ProcedureId;
SELECT OBJECT_DEFINITION(OBJECT_ID(N'dbo.GetEmployeeByEmail'))
       AS ProcedureDefinition;
```

HashState only distinguishes an empty value from a nonempty one; it does not verify that the hash is valid. Metadata may be hidden without the required SQL permissions, so a null metadata result also deserves a permission check.

14. How Data Reaches the UI

The frontend behavior below was reported earlier in the conversation. The frontend source is not part of this backend repository, so its current implementation was not independently re-inspected for this document.

Reported frontend file	Responsibility
src/redux/api/authApi.ts	POST /api/auth/login relative to VITE_API_URL or the reported localhost:5256 default.
src/types/apiTypes.ts	Login body type uses username and password.
src/pages/Login.tsx	Registers the input as username and passes submitted values directly to login.
src/lib/auth.ts	Reported storage keys: dorthy_auth_token and dorthy_auth_expires_at.
src/redux/api/baseApi.ts	Reported prepareHeaders adds Authorization: Bearer <token> to API requests.

Controller to view, step by step

1. The frontend starts a GET request when a component or query hook asks for employees.
2. The backend authenticates the bearer token and checks the action's role requirement.
3. EmployeeApiController.Get calls the service and receives employee objects.
4. Ok produces a response result. ASP.NET Core serializes the objects into JSON, excluding PasswordHash because of JsonIgnore.
5. The frontend HTTP/query library parses the response JSON and stores the resulting JavaScript values in component or query state.
6. A component renders names, emails, or other fields from that state. A later state update makes the component render again.

```
// Illustrative browser code; not an existing frontend file.
async function loadEmployees(token) {
  const response = await fetch('http://localhost:5256/api/employee', {
    headers: { Authorization: `Bearer ${token}` }
  });
  if (!response.ok) {
    throw new Error(`Employee request failed: ${response.status}`);
  }
  return await response.json();
}
```

```
// A React component could render:
// employees.map(employee => (
//   <tr key={employee.id}>
//     <td>{employee.name}</td>
//     <td>{employee.email}</td>
//   </tr>
// ))
```

The response might look like this. Values are illustrative:

```
[
  {
    "id": 12,
    "name": "Study Employee",
    "email": "learner@example.com",
    "phoneNumber": "9800000000",
    "salary": 25000,
    "departmentId": 1,
    "clientId": null,
    "role": "Employee",
    "client": null,
    "department": null
  }
]
```

Department and Client may be null with the current simple Dapper entity mapping. The presence of DepartmentId alone does not populate DepartmentName. The frontend must handle this response shape or the API must deliberately introduce a richer response DTO.

Login and navigation

After successful login, the frontend stores the returned token and decides where to navigate. The backend login endpoint returns JSON, not a redirect. The same route/layout can be used for both roles, with role-aware navigation and actions. Inspect the actual decoded role claim key; ClaimTypes.Role may appear as a mapped or URI-form claim rather than the exact property name a frontend guesses.

Decoding a token in the UI is useful for display, but the backend's signature and claim validation remains authoritative. An expired token should trigger a sign-in flow; 403 should show denied access rather than be presented as an incorrect password. A 204 response has no JSON body, so do not call `response.json()` unconditionally after every API request.

15. Setup and Test Walkthrough

This is a manual learning sequence. The document generation did not run these API calls, apply migrations, send mail, or modify SQL Server. Use a development database and mail recipients you control.

Step 1: confirm prerequisites

You need a compatible .NET SDK, a reachable SQL Server, the intended connection configuration, and all six employee procedures. SSMS is helpful for inspecting procedures but is not required by Dapper itself.

```
dotnet --info
dotnet restore EmployeeApi.sln
dotnet build EmployeeApi.sln

# Starting the app applies pending migrations through its initializer.
dotnet run --project EmployeeApi --launch-profile http
```

The current http launch profile uses `http://localhost:5256` and Development. Swagger is normally available at `http://localhost:5256/swagger`. Confirm the startup log in case you override the URL. If the executable is locked during a build, stop the relevant backend/debug session before rebuilding.

Step 2: obtain a Manager token

In Postman, create a POST request to `http://localhost:5256/api/auth/login` with a raw JSON body. Use the configured `ApiUser` credentials from your effective local configuration, or a valid database employee whose role is Manager. This document does not reproduce those credentials.

```
{
  "username": "YOUR_CONFIGURED_MANAGER_LOGIN",
  "password": "YOUR_CONFIGURED_MANAGER_PASSWORD"
}
```

A successful result is 200 with token and expiresAt. In Postman's Authorization tab choose Bearer Token and supply the token. Do not duplicate the word Bearer inside a field that already adds that prefix.

Step 3: create a department

```
POST /api/department
Authorization: Bearer <manager-token>
Content-Type: application/json

{
  "name": "Engineering",
  "description": "Study department"
}
```

Expect 201 and capture the returned Id. Use that real value as departmentId in the employee request. You can leave clientId null, or create a Client first if you want to study that relationship.

Step 4: optionally create a client

```
POST /api/client
Authorization: Bearer <manager-token>
Content-Type: application/json

{
  "clientName": "Study Client",
  "phoneNumber": "9800000000",
  "projectName": "Employee Portal"
}
```

Expect 201 and use the returned Id if assigning the employee to this client.

Step 5: create the employee

```
POST /api/employee/new
Authorization: Bearer <manager-token>
Content-Type: application/json

{
  "name": "Study Employee",
  "email": "learner@example.com",
  "phoneNumber": "9800000000",
  "salary": 25000,
  "departmentId": 1,
  "clientId": null,
  "password": "StudyOnly123!",
  "role": "Employee"
}
```

Replace departmentId with the actual returned ID. For real email testing, replace the example address with an inbox you control. With mail disabled, expect 201 after persistence. With mail enabled, SMTP submission must also succeed before this controller returns 201. The response must not contain passwordHash or the raw password.

Step 6: log in as that employee

```
POST /api/auth/login
Content-Type: application/json

{
  "username": "learner@example.com",
  "password": "StudyOnly123!"
}
```

Use the same email and password as creation. Store this token separately from the Manager token so authorization tests do not accidentally reuse a Manager identity.

Step 7: verify authorization

Test	Expected with valid data/configuration
No token: GET /api/employee	401
Employee token: GET /api/employee	200
Employee token: POST /api/employee/new	403
Employee token: GET /api/assign-task	403
Employee token: POST /api/department with valid body	Currently allowed; normally 201
Employee token: POST /api/client with valid body	Currently allowed; normally 201
Manager token: GET missing employee ID	404
Wrong employee password: login	401

Step 8: update profile data

```
PUT /api/employee/12
Authorization: Bearer <manager-token>
Content-Type: application/json

{
  "name": "Study Employee Updated",
  "email": "learner@example.com",
  "phoneNumber": "9800000000",
  "salary": 28000,
  "departmentId": 1,
  "clientId": null
}
```

Use actual IDs. Expect 200 with the new profile values. Confirm that the previous password still works and the previous role remains. Adding password and role to this body currently does not change them.

Step 9: assign a task

```
POST /api/assign-task
Authorization: Bearer <manager-token>
Content-Type: application/json

{
  "employeeId": 12,
```

```
"title": "Study the login flow",
"description": "Trace the controller and stored procedure.",
"dueDate": null,
"status": "Pending"
}
```

Expect 201 for an existing employee. GET /api/assign-task/employee/12 retrieves that employee's assignments for a Manager. PUT /api/assign-task/{id} uses the same body fields; verify that AssignedOn is preserved when editing.

Step 10: test deletion deliberately

Delete test assignments first, then the test employee, then optional test clients/departments after checking references. Use actual returned IDs. DELETE /api/employee/{id} should return 204 once, then 404 on a second deletion attempt. Avoid deleting a shared department merely to clean up one employee because the department relationship can cascade.

Step 11: test email failure separately

In a development environment, verify Enabled=false skips mail. Then enable correct SMTP settings and create a fresh test account. Inspect the inbox and spam folder. To study failure handling, observe a controlled provider/network failure and check whether the row was already saved before attempting any retry. Do not mistake a 500 response for proof that no data was inserted.

Existing Postman artifacts

The repository includes Postman/EmployeeApi.postman_collection.json and Postman/EmployeeApi.postman_environment.json. They can be imported for convenience. Verify their URLs, bodies, and auth configuration against the routes in this document; their contents were not used as the authority for the current access matrix.

16. Troubleshooting and Gaps

Follow the failing boundary

Symptom	Where to inspect and why
Could not find stored procedure	EmployeeService command name, effective DefaultConnection, SSMS database and schema, procedure deployment, and SQL identity permissions. The final line of a multi-line await call can be highlighted because that is where the exception is observed.
ApiUser works but employee login fails	The configured branch does not call the employee procedure. Inspect GetByEmail, email value, returned PasswordHash and Role, and actual database row.
Invalid username or password	Send email under username; confirm the account and hash in the same database. Do not compare plaintext against PasswordHash in SQL.
403 after successful login	Check the role in the token and the action/controller Authorize requirements. Use a freshly issued token after a database role change.
401 on protected requests	Check the Authorization header, expiry, issuer, audience, signature key, and whether the correct environment issued the token.
Create throws because no row was returned	dbo.CreateEmployee must SELECT exactly one inserted employee row for QuerySingleAsync. INSERT alone is insufficient.
Update loses role or login capability	GetById must return the existing Role and PasswordHash before the controller copies them. Verify SQL column names and mapping.
Delete result is wrong	dbo.DeleteEmployee must SELECT the immediate deletion count for ExecuteScalarAsync. SQL RETURN is a different mechanism.
Null PhoneNumber insert failure	The create/update requests allow null, but the entity/snapshot expects a non-null column. Normalize in a deliberately chosen C# or SQL boundary; the teaching SQL uses COALESCE.

Symptom	Where to inspect and why
Foreign key error	Check DepartmentId, ClientId, and dependent rows before insert/update/delete. HTTP validation does not confirm relationship existence.
Email authentication or TLS failure	Check MailSettings credentials, provider policy, authorized sender, and STARTTLS port. An employee password is not an SMTP credential.
500 on employee creation, but row exists	Inspect SMTP failure after the committed Create call. Check for an existing row before retrying.
Department object is null in JSON	Simple Dapper mapping does not load navigation objects. Check expected API response shape rather than assuming Include behavior survived the conversion.
Works in Postman but browser reports CORS	Inspect browser request origin, API URL, preflight, and CORS middleware/configuration. Postman does not enforce browser CORS rules.
App fails before Swagger loads	Check startup migration errors, SQL connectivity, missing JWT configuration, and startup logs.

Missing-procedure investigation, step by step

1. Place a breakpoint after EmployeeService reads the connection string. Inspect server/database locally without publishing the credential-bearing string.
2. Remember that environment variables or another environment-specific file may override appsettings.json.
3. In SSMS, select the same database and run `SELECT @@SERVERNAME, DB_NAME()`.
4. Inspect sys.procedures for the exact schema and name.
5. Execute `dbo.GetEmployeeByEmail` with a test email in that database.
6. If SSMS succeeds under a different SQL identity, verify the application's identity can see/execute the object.
7. If the definition is missing, deploy it through the chosen SQL or migration process; rebuilding C# will not create it.

Current implementation gaps worth understanding

These are observations and future edit locations, not changes made for this document.

Observation	Relevant files and implication
Update Password and Role inputs are unused	EmployeeApiController.Update and UpdateEmployeeViewModel. The API contract suggests optional changes that the action currently ignores.
Department/Client writes allow Employee	DepartmentApiController and ClientApiController class attributes. Change them if this differs from the intended permission rules.
Employee reads are unrestricted by ownership	EmployeeApiController Get/GetById and the procedure contracts. Both roles can obtain employee rows including salary; decide what each role should see.
Assigned tasks have no mapped employee foreign key	AssignedTask, ApplicationDbContext, and snapshot. The service existence check alone cannot guarantee referential integrity.
Roles are arbitrary strings	CreateEmployeeViewModel.Role and create mapping. Validate a defined allowlist before token issuance uses stored roles.
Email uniqueness is not configured in the inspected EF model	ApplicationDbContext and GetByEmail contract. Add deliberate normalization/uniqueness rules with data cleanup and schema work if required.
SMTP follows persistence in the same HTTP action	EmployeeApiController.Create and SmtEmployeeEmailSender. Delivery failure can leave a saved account and a failed request; a queue/outbox and retry design would address this more reliably.
Password is emailed and change flow is absent	SMTP body and auth/employee controllers. The email's change-password instruction is not backed by a working self-service endpoint.
Configured Manager account bypasses employee lookup	AuthApiController's ApiUser branch. It is a separate plaintext-configured login and lacks an employee ID claim; plan its lifecycle deliberately.
Token revocation/refresh and login throttling are absent	AuthApiController and Program. Existing tokens do not automatically reflect account deletion or role changes.
Some errors become unhandled server errors	Employee update races, SQL exceptions, and SMTP exceptions lack consistent HTTP error mapping. Avoid leaking diagnostic details to production clients.
Stored-procedure bodies are not versioned here	No procedure SQL files were found and addspg is empty. A fresh deployment needs a repeatable procedure deployment step.

For debugging, record route, status code, operation name, and nonsecret identifiers. Do not log raw passwords, SMTP credentials, bearer tokens, or full credential-bearing connection strings.

17. Practice Lessons

Use these exercises in order. Explain your prediction before executing anything. Database write exercises belong in a test database.

Lesson	Exercise	Answer to check afterward
1. Request flow	Trace GET /api/employee/7 through every backend layer.	EmployeeApiController.GetById -> IEmployeeService -> EmployeeService.GetById -> dbo.GetEmployeeById -> mapped employee -> Ok/NotFound -> JSON.
2. Object defaults	Why can a new Employee object become a Manager despite its default role?	Create assigns model.Role to employee.Role; a database query can also populate Role. The initializer only supplies an initial value.
3. JSON contracts	What happens if employee login sends email instead of username?	The current request type does not map an email property into Username. The lookup receives the default/missing username value.
4. Validation	Does EmailAddress prove that the employee owns the address?	No. It checks syntax, not ownership or deliverability.
5. DI	Who constructs EmployeeService for the controller?	The DI container uses AddScoped registration and resolves its constructor dependencies.
6. Persistence	Why does DepartmentService call SaveChangesAsync but EmployeeService.Create does not?	EF tracks pending changes until save; Dapper executes the INSERT procedure directly.
7. Passwords	Hash the same test password twice. Should the outputs match?	Normally no, because a fresh random salt is used. VerifyHashedPassword checks a candidate against a stored output.
8. Authorization	Why does an Employee token receive 403 on create employee?	Its identity is authenticated, but the action requires Manager.
9. SQL results	Remove the final SELECT from a study Create procedure. What breaks?	QuerySingleAsync expects exactly one result row and throws when it receives none.

Lesson	Exercise	Answer to check afterward
10. Scalar result	Can RETURN 1 replace SELECT @@ROWCOUNT for current Delete?	No. ExecuteScalar reads a result-set value, not a procedure return-value parameter.
11. Mapping	What if GetById omits Role and PasswordHash?	Defaults may be copied during update and overwrite real credentials/role. The procedure's result shape is essential.
12. Updates	Send optional password/role fields in the current update request. Predict the outcome.	The action ignores them and preserves the stored values. Validating a field does not mean applying it.
13. SMTP	Set Enabled=false and create an employee.	The row can be saved and 201 returned; the sender exits without contacting SMTP.
14. Failure order	Suppose SMTP fails after the insert. Is the insert rolled back?	No rollback is implemented across these two operations. The employee may already exist.
15. Migrations	Do you generate a migration to change a welcome-email subject?	No. There is no persisted schema change.
16. Procedures	Does the empty addspg migration create GetEmployeeByEmail?	No. It contains no SQL and no schema operations.
17. Navigation	Why can DepartmentId have a value while Department is null?	The foreign-key value was mapped, but the related object was not loaded/multi-mapped.
18. Frontend	What renders the employee's name after Ok(employee)?	The backend serializes JSON; the frontend parses it, stores it in state, and renders the property in a component.

A complete account-creation trace to explain aloud

A Manager submits JSON to POST /api/employee/new. JWT middleware validates the token, then authorization confirms Manager. ASP.NET binds and validates CreateEmployeeViewModel. The controller maps profile fields, hashes the supplied password, sets the role, and calls the service. Dapper sends named parameters to dbo.CreateEmployee. SQL inserts the row and selects it back with its new Id. Dapper maps that row into Employee. The controller invokes the email sender, which either exits when disabled or submits a message through SMTP. Finally, CreatedAtAction produces a 201 response with the employee JSON and a Location header. The frontend receives it and updates the displayed list or navigates according to its own logic.

A complete login trace to explain aloud

The employee enters their email into the login field and their password. The frontend sends username and password. AuthApiController first checks the configured ApiUser branch. Otherwise it passes username to GetByEmail, which calls the SQL procedure and returns the employee including PasswordHash and Role. The hasher verifies the candidate password. On success, CreateToken builds signed claims using the employee's email, role, and ID. The response gives the frontend a token and expiry. Later requests attach the token; middleware validates it and action attributes determine whether the requested operation is allowed.

18. Edit Map and References

Where to make a future change

Goal	Files or database objects to inspect
Add a stored employee field	Model/Employee/Employee.cs; relevant ViewModels; controller mapping; EmployeeService parameters; SQL procedures; ApplicationDbContext mapping as needed; a reviewed migration; frontend contract.
Change employee login identifier	ViewModel/Auth/LoginRequestViewModel.cs; AuthApiController.Login; IEmployeeService/EmployeeService lookup; database lookup procedure; frontend form and request type.
Change role permissions	Authorize attributes in Api/v1 controllers. Add ownership checks separately where required. Keep the frontend permission display consistent.
Change password behavior	CreateEmployeeViewModel and UpdateEmployeeViewModel; EmployeeApiController mapping; AuthApiController verification; plan a dedicated reset/change contract if needed.
Change SQL access	EmployeeService plus the deployed dbo procedure definitions. Preserve parameter names and returned result contracts.
Add employee paging	IEmployeeService.GetAll signature; EmployeeService; Get action query parameters; a paged response DTO; GetAllEmployees procedure; frontend list contract.
Change SMTP provider	MailSettings configuration values and secret store. Change SmtpEmployeeEmailSender only if the new provider needs a different transport/authentication mechanism.
Change welcome-email contents	Service/Notifications/SmtpEmployeeEmailSender.cs subject/body. Update the interface/caller only when extra inputs are required.
Add a service	New interface and implementation under Service; EmployeeApiServiceRegistrar registration; constructor injection into the consumer.

Goal	Files or database objects to inspect
Change table relationships	Models, ApplicationDbContext.OnModelCreating, a migration, data cleanup, and service deletion/validation behavior.
Change local listening URL	Properties/launchSettings.json or host URL configuration; update the frontend API base URL and Postman environment accordingly.

Glossary

Term	Plain-language meaning
Endpoint	A method-and-URL combination such as POST /api/auth/login.
DTO / ViewModel	A data contract used to carry values across an application boundary.
Entity	An object representing domain data that is mapped to persistence.
Dependency injection	Constructing a class with the services it needs instead of having it construct those dependencies itself.
ORM / mapper	A library that translates between database results and application objects. EF Core also manages a model and change tracking; Dapper focuses on explicit SQL mapping.
Stored procedure	Named SQL logic deployed inside a database and called with parameters.
Result set / scalar	A table-shaped query result / a single value read from the first column of the first row.
Foreign key	A database constraint connecting a child value to an existing parent key.
Migration / snapshot	A versioned schema-change operation / EF's saved model shape used for comparison.
Hash / salt	A derived password-verification value / random input that helps distinguish equal passwords' stored hashes.
JWT / claim	A token containing structured assertions / one named assertion such as a role.

Term	Plain-language meaning
Bearer token	A token whose holder can present it to authenticate a request.
SMTP / STARTTLS	A protocol for submitting/transferring mail / a command that upgrades an SMTP connection to TLS.
401 / 403 / 404	Authentication required or invalid / identity lacks required access / resource not found.

Sources and verification boundaries

The primary source for the application behavior is the current EmployeeApi repository: the source listings throughout this document are taken directly from its files. Source comments and commented-out legacy blocks are omitted for readability where applicable; executable statements are retained with original line numbers. No SMTP password, JWT secret, or configured ApiUser password was copied from appsettings.json.

The six procedure bodies in Chapter 8 are clearly identified teaching examples because deployed SQL definitions were not available locally. Frontend implementation notes are attributed to the earlier user-supplied report. Generated PDF checks concern document completeness and rendering, not application runtime correctness.

Framework and provider references for further reading:

- [Microsoft: role-based authorization](#) supports the role-attribute rules in Chapter 10.
- [Microsoft: PasswordHasher API](#) describes the hashing and verification methods used in Chapter 9.
- [ASP.NET Core 10 PasswordHasher source](#) shows the versioned PBKDF2 implementation, random salt, and verification behavior.
- [Dapper's official repository](#) documents parameterized mapping and stored-procedure usage.
- [Microsoft: managing EF migrations](#) describes generated migrations and custom SQL operations.
- [Microsoft: SmtpClient.EnableSsl](#) explains STARTTLS behavior and the unsupported implicit TLS connection mode.
- [Google: Gmail SMTP settings](#) lists provider connection settings.
- [Google: app passwords](#) explains setup and account restrictions.

End of study guide. Revisit the relevant source files when the project changes; this is a documented snapshot, not a replacement for current code and deployed database definitions.